

멀티 클라우드 환경에서의 LLM 기반 로그 분석 및 이상 탐지 시스템



저자 1 202155595 이한홍

저자 2 202155590 이종일

저자 3 201924479 박재홍

지도교수 김호원

목 차

1. 서론.....	1
1.1. 연구 배경.....	1
1.1.1. 클라우드 환경에서의 로그 관리 중요성.....	1
1.1.2. LLM(대형 언어 모델)을 활용한 로그 분석.....	1
1.1.3. 클라우드 환경에서의 보안 위협 증가.....	2
1.2. 기존 문제점.....	2
1.2.1. 대규모 로그 데이터 처리의 어려움.....	2
1.2.2. 전통적인 탐지 방식의 한계.....	3
1.2.3. 기존 로그 분석 도구의 실용성 문제.....	3
1.3. 연구 목표.....	4
2. 연구 배경.....	4
2.1. 배경 지식.....	4
2.1.1. Filebeat.....	4
2.1.2. ElasticSearch.....	5
2.1.3. Logstash.....	5
2.1.4. Kibana.....	5
2.1.5. LangChain.....	6
2.1.6. FastAPI.....	6
2.1.7. Spring Boot.....	6
2.1.8. React 및 Next.js.....	6
2.1.9. Apache MINA SSHD.....	7

2.1.10.	Docker.....	7
2.1.11.	Redis.....	7
2.1.12.	MySQL.....	7
2.2.	서비스 기획.....	8
2.2.1.	화면 설계	8
2.2.2.	기능 정의	8
2.2.3.	개발 환경 구성.....	11
2.2.4.	데이터베이스 설계	11
3.	연구 내용	13
3.1.	서비스 아키텍처	13
3.2.	로그의 계층적 관리.....	14
3.3.	토큰 한계를 해결하는 대화 맥락 유지 전략	15
3.4.	웹 서비스 구성.....	16
3.4.1.	사전 준비	16
3.4.2.	설치 과정	17
3.4.3.	회원 가입	18
3.4.4.	로그인	21
3.4.5.	대시보드.....	22
3.4.6.	로깅/모니터링.....	23
3.4.7.	인사이트.....	28
3.4.8.	검색.....	30
3.4.9.	알람.....	32
3.4.10.	설정.....	32
4.	연구 결과 분석 및 평가	34

4.1.	실시간 로그 수집	34
4.2.	LLM을 활용한 로그 요약 및 이상 패턴 탐지.....	34
4.3.	원격 클라우드 SSH 접속	34
4.4.	웹 기반 클라이언트 환경	35
4.5.	LLM 기반 로그 분석 비용 평가.....	35
5.	결론 및 향후 연구 방향	36
6.	구성원별 역할	37
7.	일정.....	38
8.	멘토 의견 반영.....	39
8.1.	컨셉과 시나리오가 여전히 구체적이지 않음	39
8.2.	범용성 있는 로그 분석 시스템 개발에 한계가 있을 것임.....	39
8.3.	타겟을 명확히 설정한 후 개발 진행 필요.....	39
9.	참고 문헌	40

1. 서론

1.1. 연구 배경

1.1.1. 클라우드 환경에서의 로그 관리 중요성

현대 IT 인프라에서 로그 데이터는 시스템 성능 평가, 보안 강화, 그리고 문제 발생 시 신속한 대응에 중요한 역할을 한다. 시스템 운영 중 발생하는 이벤트와 상태 정보를 기록하는 로그 데이터를 통해, 개발자와 관리자는 애플리케이션의 상태를 실시간으로 확인하고, 문제 발생 시 원인을 빠르게 파악할 수 있다.

이러한 로그 관리의 필요성은 **클라우드 컴퓨팅**의 보편화로 인해 더욱 중요해졌다. 특히, 멀티 클라우드 환경과 마이크로서비스 아키텍처(MSA)와 같은 복잡한 시스템 구조가 도입되면서, 애플리케이션이 여러 구성 요소와 인프라에 걸쳐 **분산**되었다. 그 결과, 로그 데이터를 수집하고 분석하는 과정이 점차 **복잡**해졌으며, 로그 데이터의 **양**도 기하급수적으로 증가하고 있다. 이러한 상황에서, 다양한 소스에서 생성되는 로그를 일관되게 처리하고 실시간으로 분석할 수 있는 능력은 시스템 운영의 핵심 요소가 되었다.

복잡한 클라우드 환경에서 **단순한** 로그 관리 방식으로는 시스템 운영의 신뢰성과 안정성을 보장하기 어렵다. 유연하고 확장 가능한 로그 처리 시스템을 통해, 클라우드 환경에서 발생하는 다양한 문제를 실시간으로 감지하고 대응할 수 있어야 한다.

이러한 요구로 인해, 클라우드 인프라에서 로그 데이터를 실시간으로 수집, 분석, 관리하는 능력은 시스템 운영의 핵심 요소로 자리 잡게 되었다.

1.1.2. LLM(대형 언어 모델)을 활용한 로그 분석

최근 자연어 처리(NLP) 기술의 발전은 로그 데이터 분석에 새로운 가능성을 열어주고 있다.

기존의 로그 분석 방식은 주로 **규칙 기반의 고정된 패턴 탐지**에 의존해 왔다. 하지만, 멀티 클라우드 환경이나 마이크로서비스 아키텍처(MSA)와 같은 복잡한 시스템에서는 방대한 로그 데이터가 실시간으로 생성되고, 이러한 데이터를 효율적으로 분석하기 위해서는

새로운 접근 방식이 필요하다.

LLM은 **자연어 데이터** 처리에 탁월한 성능을 발휘하며, 이를 로그 분석에 적용하면 **핵심 정보**를 빠르게 추출하고, **비정상적인 패턴**을 정확하게 탐지하는 등 고도화된 분석이 가능해진다. LLM의 도입으로 기존 정적 규칙 기반 분석의 한계를 극복하고, 시스템 전반의 로그 데이터를 복합적으로 분석해서, 핵심 문제를 빠르게 찾아내어 대응할 수 있게 된다.

1.1.3. 클라우드 환경에서의 보안 위협 증가

클라우드 환경에서 로그는 보안 위협을 탐지하고 대응하는 데 핵심적인 역할을 한다. 다양한 어플리케이션과 서비스가 분산 운영되면서, **로그 데이터**를 통해 침입 시도나 비정상적인 동작을 조기에 **탐지**하고, 이에 신속하게 **대응** 할 수 있는 역량이 중요해졌다.

특히, 클라우드 인프라는 물리적 서버보다 접근 제어가 복잡하고, 네트워크 경로가 다양해졌으며, 여러 클라우드 서비스 간 데이터 전송이 빈번하게 이루어지면서 **보안 위협이 증가**하고 있다. 이 때문에, 클라우드 인프라의 복잡성을 고려한 체계적인 보안 전략 수립이 필수적이다.

본 연구에서 제안하는 시스템은 이러한 요구에 맞춰, 로그를 실시간으로 모니터링하고 분석함으로써, 보안 위협에 신속하게 대응할 수 있는 환경을 구축하는 것을 목표로 한다.

1.2. 기존 문제점

1.2.1. 대규모 로그 데이터 처리의 어려움

현대의 분산 시스템은 멀티 클라우드 환경과 마이크로서비스 아키텍처(MSA)를 통해 각기 다른 서비스에서 방대한 양의 로그 데이터를 생성한다. 하지만 기존 방식으로는 이러한 데이터를 실시간으로 수집, 저장, 분석하는 과정에서 **성능 저하**와 **리소스 부족** 문제가 자주 발생한다.

또한, 기존의 로그 분석 시스템은 **확장성의** 한계로 인해 대규모 로그 데이터를 관리하는 것에 어려움을 겪고, 실시간 데이터 분석을 원활하게 지원하지 못하는 경우가 많다.

그 결과 **지연**이 발생하게 되고, 문제 상황에 대한 즉각적으로 대응이 어려워진다.

1.2.2. 전통적인 탐지 방식의 한계

시스템에서 발생하는 문제를 로그를 통해 **빠르고 정확하게** 탐지하는 것은 시스템 안정성 확보에 중요한 역할을 한다.

그러나 기존의 규칙 기반 탐지 방식은 사전에 **설정된 패턴**이나 **조건에만 의존**하여, 새로운 유형의 문제나 비정상적인 동작을 탐지하는 데 한계가 있다. 예를 들어, 규칙 기반 탐지는 미리 정의된 조건에 맞는 이벤트만 감지하기 때문에, 예상치 못한 문제나 새로운 보안 위협이 발생한 경우 이를 감지하지 못해 탐지에서 누락될 수 있다.

또 이러한 방식은 **정적인** 규칙에 의존하기 때문에, 시스템 및 애플리케이션의 복잡성이 증가할수록 **다양한 시나리오**를 포괄하는 규칙을 만드는 것이 비효율적이고 복잡해진다. 이러한 한계로 인해, 오탐지(false positives)와 미탐지(false negatives)의 발생 빈도가 높아질 수 있다.

1.2.3. 기존 로그 분석 도구의 실용성 문제

많은 로그 분석 도구가 제공하는 결과물은 관리자나 개발자가 문제 해결에 직접적으로 활용하기 어려운 경우가 많다. 로그 데이터에서 도출된 정보가 지나치게 **방대**하거나, 지나치게 **단순화**되어 중요한 정보를 놓치는 경우가 있으며, 이 때문에 문제의 근본 원인을 파악하는 데 어려움이 발생할 수 있다.

따라서, 보다 **직관적**이고 핵심 정보를 신속히 제공할 수 있는 분석 도구가 필요하고, 이를 통해 관리자가 문제 해결에 집중할 수 있도록 지원해야 한다.

1.3. 연구 목표

본 연구의 목표는 멀티 클라우드 환경에서 발생하는 대규모 로그 데이터를 효율적으로 수집하고, **LLM**을 활용해 이를 요약하고 분석하는 시스템을 구축하는 것이다.

이 시스템은 실시간 로그 모니터링을 통해 주요 이벤트를 **포착**하고, 이상 패턴을 신속히 **탐지**하여, 관리자가 로그 데이터를 **일일이** 분석하지 않고도 핵심 정보를 파악할 수 있게 돕는다.

특히, 로그 데이터를 계층적으로 요약하고 **중요도**에 따라 정보를 **선별적**으로 제공함으로써, 관리자가 핵심 정보에 **집중**하여 시스템을 보다 효율적으로 모니터링하고 파악할 수 있게 한다. 또한, LLM 기반 분석으로 도출된 **구체적인 대응 방안**을 관리자에게 제시하여, 비정상적인 시스템 동작에 신속히 대응할 수 있도록 지원한다.

이 시스템을 통해 관리자는 대규모 로그 데이터 분석에 수반되는 부담을 덜고, 실시간으로 발생하는 문제를 보다 신속하고 효율적으로 해결할 수 있을 것이다.

2. 연구 배경

2.1. 배경 지식

2.1.1. Filebeat

Filebeat는 **경량 로그 수집 도구**로, 서버에서 발생하는 로그 파일을 실시간으로 수집하여 Logstash나 Elasticsearch로 전송하는 역할을 한다. 또한, 다양한 로그 데이터를 **중앙**에서 통합 관리할 수 있어서, 멀티 클라우드 환경과 마이크로서비스 아키텍처에서 발생하는 로그를 효율적으로 다룰 수 있다. 이를 통해 분산된 환경에서 발생하는 로그 데이터를 빠르고 안정적으로 수집할 수 있다.

2.1.2. Elasticsearch

Elasticsearch는 분산형 오픈 소스 **검색 엔진**으로, 대규모 데이터를 빠르게 저장하고 검색할 수 있는 기능을 제공한다. JSON 형식으로 데이터를 저장하고, 다양한 쿼리와 필터링을 지원하여 복잡한 데이터 분석 작업을 효율적으로 수행할 수 있다.

본 연구에서는 Elasticsearch를 활용해 수집된 로그 데이터를 저장하고, 클라우드 환경에서 생성되는 다양한 로그를 인덱싱하여, 고성능 검색을 통한 실시간 분석을 가능하게 한다.

2.1.3. Logstash

Logstash는 다양한 소스에서 데이터를 수집하고, 이를 변환하여 Elasticsearch로 전송하는 **데이터 파이프라인** 도구이다. 데이터를 **필터링**하고 **변환**하여 원하는 형태로 전처리함으로써, 수집된 데이터를 효율적으로 관리하고 분석할 수 있다.

또한, 다양한 플러그인을 제공하여, 클라우드 환경에서 생성되는 다양한 유형의 데이터를 유연하게 다룰 수 있다.

2.1.4. Kibana

Kibana는 Elasticsearch에서 수집된 데이터를 **시각화**하고 모니터링할 수 있는 오픈 소스 분석 도구이다. 데이터를 그래프나 차트 형태로 시각화하여 제공함으로써, 사용자가 데이터를 **직관적**으로 파악할 수 있다.

또한, 검색, 필터링, 알림 설정 등의 다양한 기능을 제공하여, 데이터를 실시간으로 분석하고 문제를 빠르게 파악할 수 있도록 지원한다.

2.1.5. LangChain

LangChain은 **자연어 처리 파이프라인**을 구성하는 데 사용되는 오픈 소스 라이브러리로, OpenAI API와 연동하여 복잡한 질의응답 시스템을 구축할 수 있다. 본 연구에서는 프롬프트 생성과 텍스트 전처리, 그리고 LLM과의 연동을 위해 LangChain을 사용하였다.

2.1.6. FastAPI

FastAPI는 **자연어 처리** 작업과 같은 복잡한 비동기 요청을 효율적으로 처리할 수 있는 **파이썬 기반 웹 프레임워크**로, REST API를 쉽게 구현할 수 있는 장점을 가진다. 본 연구에서는 FastAPI를 활용하여 LangChain과 OpenAI API를 연동해, 로그 데이터 분석 및 요약 기능을 구현하였다.

2.1.7. Spring Boot

Spring Boot는 **자바 기반 웹 프레임워크**로, REST API와 같은 서버 애플리케이션을 개발할 수 있는 도구이다. 본 연구에서는 Spring Boot를 활용하여 클라우드에서 생성된 로그 데이터를 Elasticsearch에서 조회하고, 이를 저장하고 관리하는 기능을 구현하였다. 또한, 사용자 인증, 프로젝트 관리, 컨테이너 제어, 알람 처리 등의 기능을 API로 제공하여, 관리자가 클라우드 환경의 시스템을 간편하게 제어하고 모니터링할 수 있는 환경을 구축하였다.

2.1.8. React 및 Next.js

React는 자바스크립트 기반의 **사용자 인터페이스(UI)** 라이브러리로, 컴포넌트 기반 개발 방식을 통해 효율적이고 재사용 가능한 UI를 구현할 수 있다.

Next.js는 React 기반의 **서버 사이드 렌더링(SSR)** 프레임워크로, 본 연구에서는 클라이언트가 로그 데이터와 분석 결과를 직관적으로 확인할 수 있도록, 대시보드 화면 구현에 이를 활용하였다.

2.1.9. Apache MINA SSHD

Apache MINA SSHD는 **SSH 프로토콜**을 통해 **원격 서버**와의 안전한 연결을 지원하는 오픈 소스 라이브러리이다. 본 연구에서는 Apache MINA SSHD를 활용하여 관리자가 웹 대시보드에서 클라우드 인스턴스에 **원격 접속**하고, 필요한 명령을 실행할 수 있는 기능을 구현하였다. 이를 통해 관리자는 시스템 상태를 원격으로 확인하고, 문제 발생 시 신속하게 대응할 수 있다.

2.1.10. Docker

Docker는 애플리케이션을 **컨테이너**라는 독립된 환경에서 실행할 수 있도록 해주는 플랫폼으로, 이를 통해 애플리케이션을 손쉽게 배포하고 관리할 수 있다. 본 연구에서는 클라우드 상에서 애플리케이션의 로그를 수집하고 분석하기 위해, Docker 기반의 구조를 활용하였다. 아울러, 본 시스템 역시 컨테이너화되어 제공되므로, 설치가 간편하고 환경 설정 관리가 쉬워진다.

2.1.11. Redis

Redis는 **인메모리 데이터베이스**로 캐시 및 메시지 브로커 등의 역할을 수행하고, 빠른 성능과 확장성을 제공한다. 본 연구에서는 Redis를 활용해 데이터를 캐싱하고, 실시간으로 로그 데이터에 대해 질문과 응답을 주고받을 수 있는 챗봇 기능을 구현하였다.

2.1.12. MySQL

MySQL은 데이터를 테이블 기반으로 저장하고 관리할 수 있는 **관계형 데이터베이스 관리 시스템(RDBMS)**이다. 본 연구에서는 알람, 유저 정보, 로그 요약 결과, 성능 지표 등을 저장하기 위해 이를 사용했다.

2.2. 서비스 기획

2.2.1. 화면 설계

각 페이지의 구조와 상호작용을 정의하여 UI/UX 디자인을 설계한다. 사용자가 필요한 정보를 신속하게 접근할 수 있도록 하여, 시스템 운영의 효율성을 극대화할 수 있게 설계되었다.

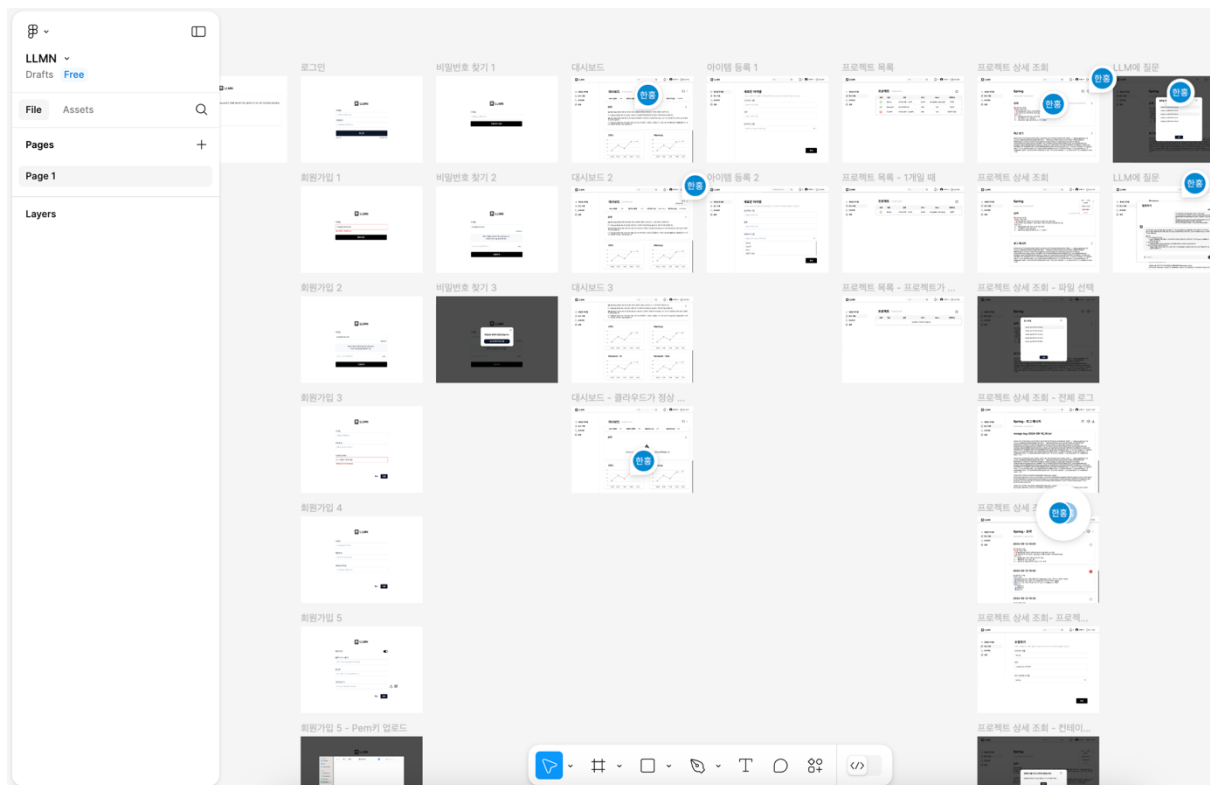


그림 1. Figma를 사용해 설계한 UI/UX 디자인

2.2.2. 기능 정의

시스템의 주요 기능을 구현하기 위해 앞서, 각 기능별 API 엔드포인트를 정의한다. 각 엔드포인트는 URL 경로와 HTTP 메소드를 명확하게 지정하여, 데이터 요청 및 응답이 효율적으로 이루어지도록 설계되었다.

번호	기능	URL	Method
1	로그인	/auth/login	POST
2	이메일 중복 체크	/accounts/check/email	POST
3	발송한 코드 검증	/accounts/verify/code	POST
4	회원가입	/accounts	POST
5	SSH 연결 유효성 체크	/accounts/validate/cloud	POST
6	OpenAI API Key 유효성 체크	/accounts/validate/key	POST
7	SSH Pem 키 업로드	/accounts/ssh	POST
8	계정 복구를 위해 이메일 코드 전송	/accounts/recovery/code	POST
9	코드 검증 후, 새 비밀번호 설정	/accounts/recovery/reset	POST
10	인증 코드 재발송	/accounts/resent/code	POST
11	대시보드 조회	/home	GET
12	모니터링 가능한 클라우드 목록 조회	/cloud	GET
13	모니터링 할 클라우드 변경	/cloud	PATCH
14	프로젝트 생성	/project	POST
15	클라우드에서 실행중인 컨테이너 목록 조회	/project/cloud	GET
16	프로젝트 목록 조회	/project	GET
17	프로젝트 목록 새로고침	/project/refresh	GET
18	프로젝트 상세 조회	/project/{projectId}	GET
19	프로젝트의 로그 파일 목록 조회	/project/{projectId}/logs	GET
20	프로젝트의 특정 로그 전체 조회	/project/{projectId}/logs/{logFileName}	GET
21	특정 로그에 대해 LLM에 질문하기	/logs/question	POST
22	로그 파일 다운	/logs/{logFileName}/download	GET
23	프로젝트의 요약 조회	/project/{projectId}/logs/summaries	GET
24	프로젝트 수정하기	/project/{projectId}	PATCH

25	프로젝트 정보 조회 (수정 용)	/project/{projectId}/info	GET
26	도커 컨테이너 재시작	/project/{projectId}/container/restart	POST
27	도커 컨테이너 종료	/project/{projectId}/container/stop	POST
28	프로젝트 삭제하기	/project/{projectId}	DELETE
29	요약에 확인 체크하기	/summaries/{summaryId}/check	POST
30	인사이트 목록 조회	/insight	GET
31	성능요약 목록 조회	/insight/performance	GET
32	일일 요약 목록 조회	/insight/daily	GET
33	장기 트렌드 조회	/insight/trend	GET
34	추천 작업 조회	/insight/recommendation	GET
35	알람 목록 조회	/alarms	GET
36	알람 읽기 처리	/alarms/read	POST
37	셸 커맨드	/shell	POST
38	유저 설정 정보 조회	/accounts/info	GET
39	유저 설정 변경	/accounts	PATCH
40	API 키 변경	/accounts/apiKey	PATCH
41	회원 탈퇴	/accounts/withdraw	DELETE
42	검색	/search	GET

2.2.3. 개발 환경 구성

시스템 요구사항을 충족하기 위해 기술 스택과 도구들을 선정하고, 이를 기반으로 개발 환경을 체계적으로 구성한다.

번호	구분	버전
1	Filebeat	8.5
2	ElasticSearch	8.15
3	Logstash	8.15
4	Docker	24.0.5
5	Redis	7.24
6	MySQL	8.0.33
7	Java JDK	17.0.9
8	Spring Boot	3.2.0
9	JWT	4.3.0
10	JUnit5	5.10.2
11	FastAPI	0.110.3
12	Gradle	8.5
13	React	16.14.0
14	TypeScript	5.6
15	NextJS	12
16	Tailwind CSS	v3.4.3
17	Apache MINA SSHD	2.93
18	Figma	UI3

2.2.4. 데이터베이스 설계

데이터의 효율적인 저장과 조회를 위해 데이터베이스 구조와 테이블 간 관계를 설계하고, 일관성을 확보하기 위해 ERD 를 작성한다.

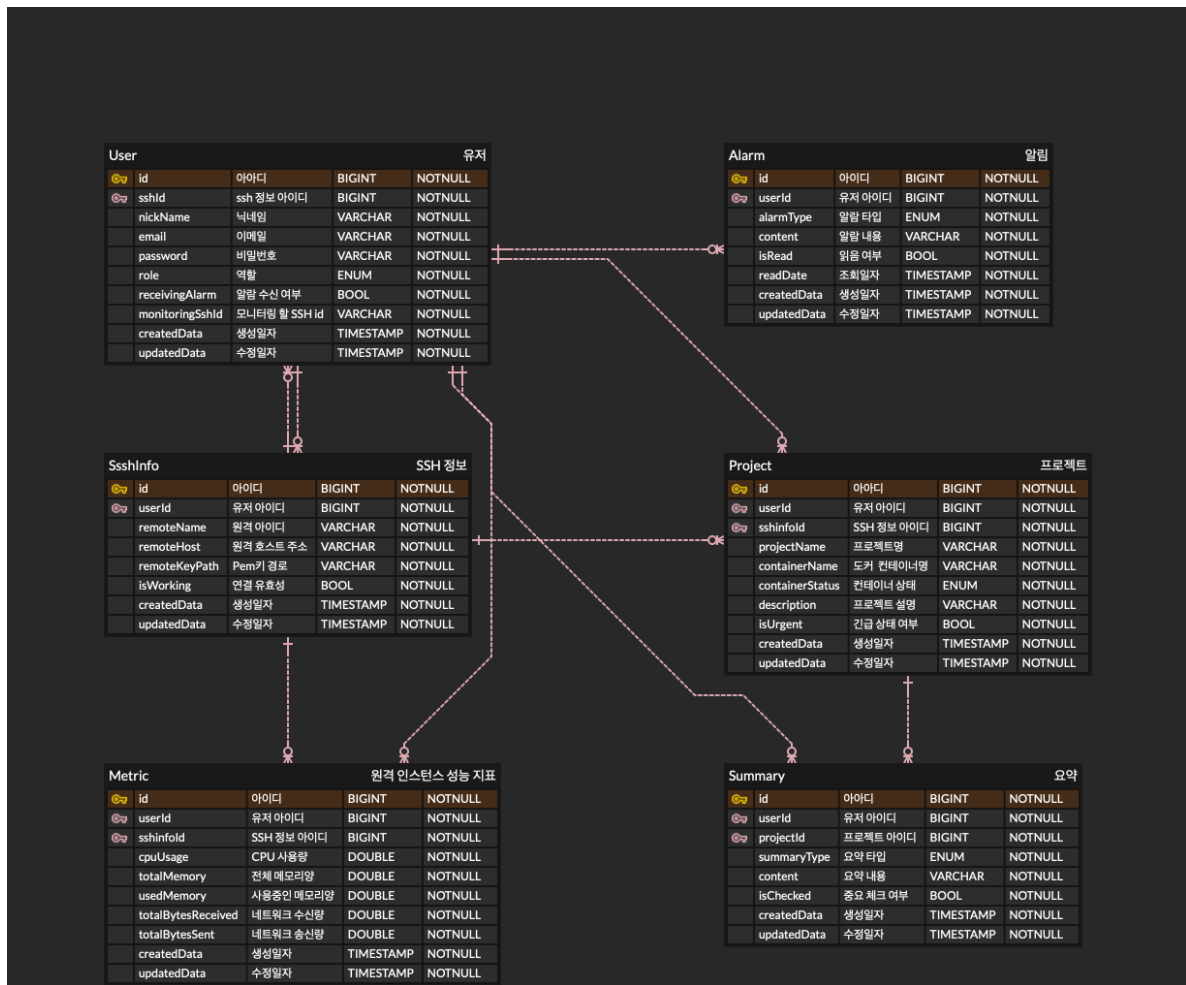


그림 2. ERD 다이어그램

번호	테이블 명	설명
1	User	사용자의 인증 정보와 권한을 저장한다.
2	SshInfo	원격 호스트에 대한 SSH 접속 정보를 저장하여, 관리자가 클라우드 인스턴스에 원격으로 접속할 수 있도록 한다.
3	Alarm	새로운 이벤트 발생 시 알림 정보를 저장한다.
4	Project	클라우드에서 컨테이너 형태로 운영되는 애플리케이션의 운영 상태와 관련 정보를 저장한다.
5	Metric	각 클라우드 인스턴스의 CPU, 메모리, 네트워크 등의 성능 지표를 저장한다.
6	Summary	로그 및 시스템의 요약 정보를 저장한다.

3. 연구 내용

3.1. 서비스 아키텍처

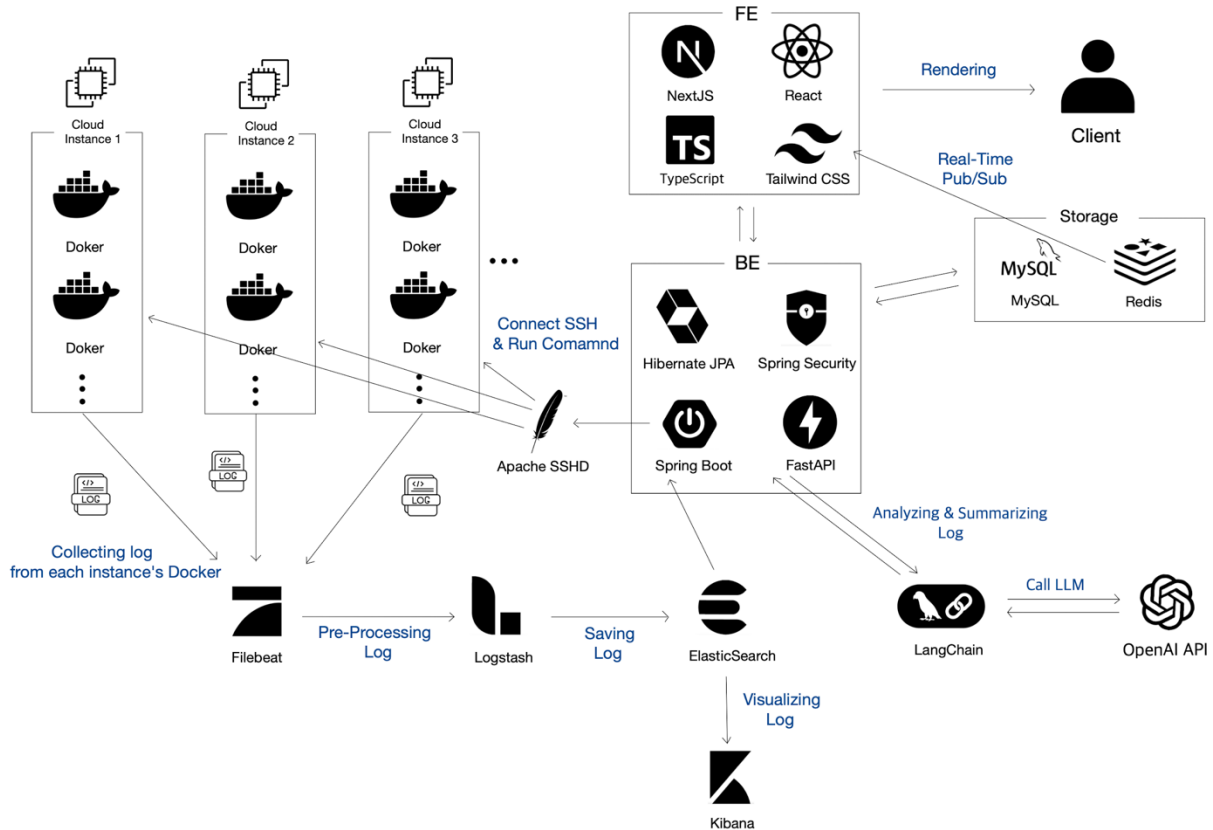


그림 3. 서비스 아키텍처

본 시스템은 **멀티 클라우드 환경**의 각 인스턴스별 로그 데이터를 실시간으로 **수집**하고 **분석**하여, 관리자에게 웹 인터페이스 형태로 제공하는 구조로 설계되었다.

클라우드 인스턴스에서 애플리케이션은 **도커 컨테이너**로 구동되고, 각 컨테이너에서 생성된 로그는 Filebeat를 통해 Logstash로 전송된 후, 전처리 과정을 거쳐 Elasticsearch에 저장된다. 저장된 로그는 Spring Boot를 통해 로드되어, 최종적으로 .txt 파일 형식으로 로컬에 저장된다.

백엔드는 Spring Boot와 FastAPI를 기반으로 구축되었으며, 데이터 저장소로는 MySQL과 Redis가 사용되었다. 사용자 정보, 알림, 요약 결과, 성능 지표 등의 주요 데이터는 MySQL에 저장되고, Redis는 캐시 및 메시지 브로커로 기능하여 클라이언트와 서버 간 실시간 소켓 통신을 지원한다.

로그 데이터 **분석과 요약**에는 LangChain과 OpenAI API가 사용되고, 분석 결과는 웹 클라이언트를 통해 관리자에게 전달된다. 프론트엔드는 React와 Next.js로 구현되어, 사용자에게 직관적이고 시각적으로 명확한 분석 결과를 제공한다.

한편, 관리자는 Apache MINA SSHD를 이용해 클라우드 인스턴스에 **원격으로 접속**하고 명령을 실행할 수 있다. 이 기능을 통해 관리자는 실시간으로 시스템 상태를 점검하고, 필요한 조치를 신속하게 취할 수 있다.

3.2. 로그의 계층적 관리

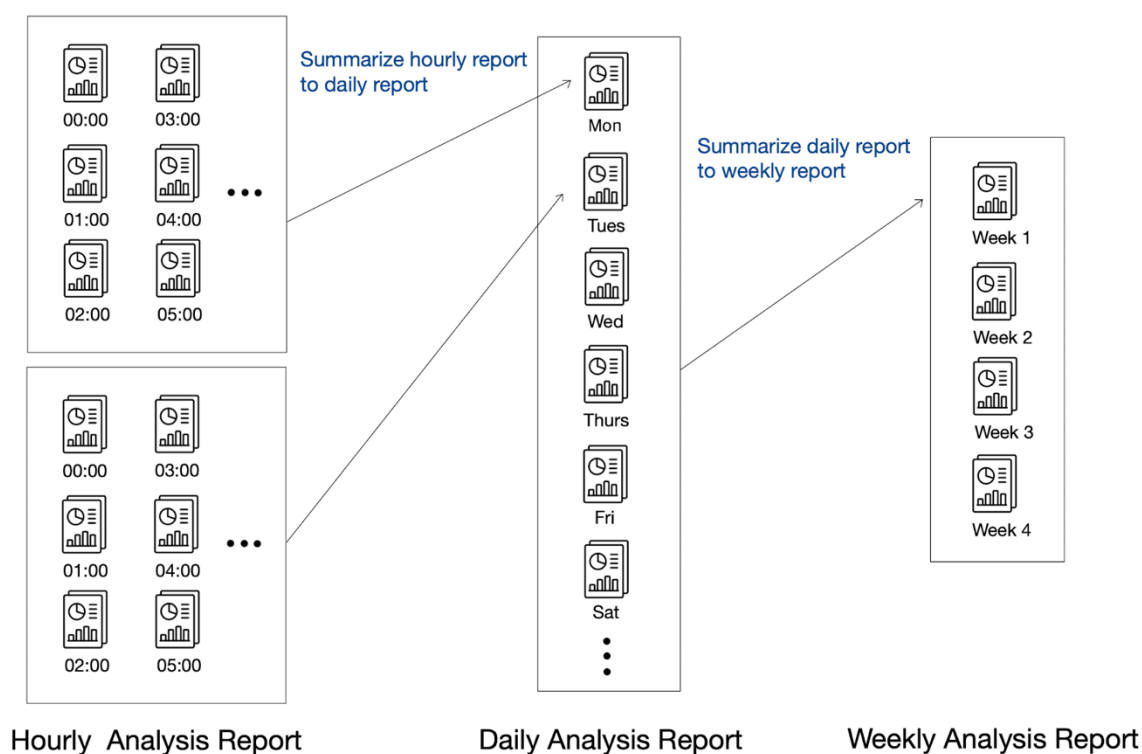


그림 4. 레이어 단위의 로그 데이터 관리 기술

본 시스템은 **대규모** 로그 데이터를 효율적으로 요약하고 관리하는 데 중점을 두고 설계되었다.

모든 로그 데이터를 그대로 **저장**하고 처리하는 대신, 각 클라우드 인스턴스에서 수집된 로그를 시간대별로 분류하고 요약하여, 해당 시간대의 **주요 이벤트**를 **단계적으로 축적**하는 방식으로 관리한다. 이를 통해 방대한 로그 데이터 중 **핵심 정보**만을 저장하여 저장

공간의 효율성을 높이고, 분석 과정에서의 불필요한 리소스 낭비를 줄일 수 있다.

또한, **모든** 로그 데이터를 LLM에 **입력**하는 것은 비효율적이므로, 시간대별로 요약된 데이터를 기반으로 일일 요약을 생성하고, 이를 종합해 주간 요약을 생성하는 **다단계 분석 방식**을 도입하였다. 이를 통해 LLM은 주요 정보를 우선적으로 처리하고, 반복적인 패턴과 이상 징후를 더 빠르고 효율적으로 탐지할 수 있게 된다.

이러한 관리 방식을 통해 관리자는 시간 단위의 세부 이벤트부터 주간 단위의 장기 트렌드까지 체계적으로 파악하여 신속하게 대응할 수 있다. 이렇게 하여, 실시간 모니터링부터 장기적 패턴 분석까지 **일관된** 결과를 제공하고, 로그 데이터 관리를 최적화하여 시스템 운영의 효율성을 향상시킨다.

3.3. 토큰 한계를 해결하는 대화 맥락 유지 전략

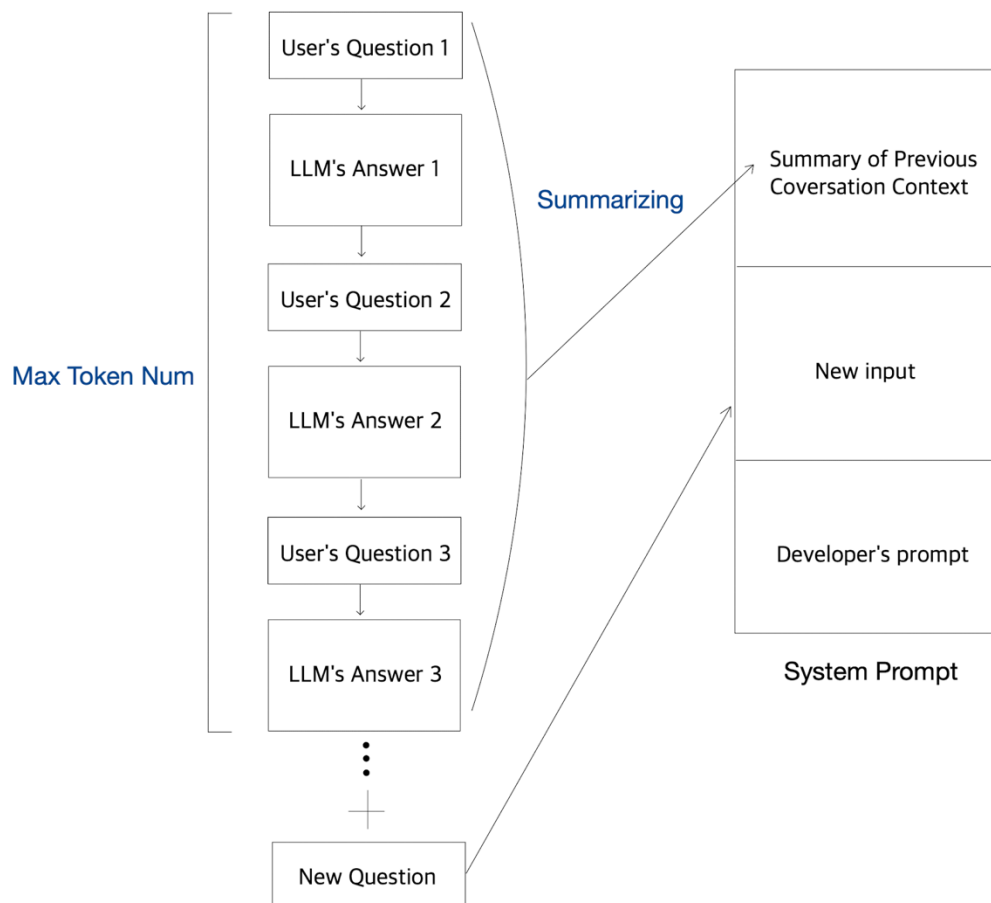


그림 5. 대화 맥락 저장 기술

본 시스템은 사용자가 특정 로그에 관해 LLM과 지속적으로 대화를 이어나갈 수 있도록, 대화 맥락 저장 기술을 도입하였다. 대화가 길어질수록 LLM의 **최대 토큰 수** 제한으로 인해, 이전 대화의 맥락을 유지하기가 점점 더 어려워진다. 이 문제를 해결하기 위해, 시스템이 대화가 일정량 누적될 때마다 이전 대화를 요약해 시스템 프롬프트에 자동으로 **포함**시키는 방식을 구현하였다.

즉, 질문과 답변이 일정량 이상 **누적**될 경우, 시스템은 이를 **요약**하여 "이전 대화 요약"에 자동으로 저장한다. 이렇게 요약된 내용은 새로운 질문과 함께 시스템 프롬프트에 포함되고, 이를 통해 LLM은 이전 대화의 주요 맥락을 유지하면서 새로운 질문에 답변할 수 있다.

이 기술을 도입한 이유는, **제한된 토큰 수** 내에서 대규모 로그 데이터를 효율적으로 처리하고, 대화의 연속성을 보장하기 위함이다. **모든 대화** 맥락을 그대로 저장하고 처리하는 것은 **비효율적**이므로, 핵심 정보를 요약하여 저장함으로써 시스템 자원의 낭비를 줄이고, 관리자가 로그 데이터에 대해 지속적으로 질의응답을 이어갈 수 있도록 한다.

이로써 대화가 길어져도 맥락이 유지된 상태에서 답변을 받을 수 있고, 토큰 수 제한으로 인해 대화 흐름이 끊기는 것을 방지할 수 있다.

3.4. 웹 서비스 구성

3.4.1. 사전 준비

서비스를 이용하기 전, 클라우드 인스턴스에서 어플리케이션이 **도커 컨테이너** 형태로 실행되고 있어야 하고, ElasticSearch가 외부와 통신할 수 있도록 9200 포트에 대한 인바운드 및 아웃바운드 트래픽을 허용해야 한다.

또한, 로그 수집을 위해 Filebeat, Logstash, ElasticSearch의 설치가 필요하다. 설치 과정을 간소화하고자, 본 시스템은 필요한 설정 파일들이 포함된 ELK 디렉토리와 docker-compose.yml 파일을 제공하여, **단일 명령어**로 간편하게 설치를 완료할 수 있도록 구성하였다.

```
docker compose up -d
```

```

1 input {
2   beats {
3     port => 5044
4   }
5 }
6
7 # 입력된 데이터를 처리하고 필터링하는 블록
8 filter {
9   # UTC 타임스탬프를 KST로 변환
10  date {
11    match => ["@timestamp", "ISO8601"]
12    timezone => "Asia/Seoul"
13    target => "@timestamp"
14  }
15
16  # 필요 없는 필드 제거
17  mutate {
18    remove_field => [
19      "input", "@version", "tags", "log",
20      "[container][labels]", "[container][image]", "[container][id]",
21      "host", "ecs", "agent", "stream"
22    ]
23  }
24 }
25 output {
26   # 데이터를 Elasticsearch로 출력하는 플러그인
27   elasticsearch {
28     hosts => ["http://elasticsearch:9200"] # 데이터를 전송할 Elasticsearch의 호스트 주소를 지정
29     index => "docker-logs-%{+YYYY.MM.dd}" # 데이터를 저장할 Elasticsearch 인덱스를 지정
30   }
31 }

```

그림 6. Logstash 설정을 위한 logstash.conf 파일

```

1 filebeat.inputs:
2   - type: container
3     paths:
4       - /var/lib/docker/containers/*//*.log
5     processors:
6       - add_docker_metadata: ~
7
8 output.logstash:
9   hosts: ["logstash:5044"]

```

그림 7. Filebeat 설정을 위한 filebeat.yml 파일

3.4.2. 설치 과정

서비스를 이용하려면, 제공된 docker-compose.yml 파일을 사용하여 LLMN 어플리케이션을 설치할 수 있다.

```
docker compose up -d
```

설치는 로컬 환경 또는 클라우드 환경 모두 가능하나, 지속적인 모니터링과 분석을 위해 **클라우드**에 설치하는 것을 권장한다. 권장 사양은 2GB 이상의 RAM이다.

3.4.3. 회원 가입



이메일

hoyai@naver.com

재전송하기

해당 이메일로 인증코드를 전송하였습니다.
아래에 인증코드를 입력해주세요.

인증 코드를 입력해주세요. 2:48

인증하기

그림 8. 회원가입 페이지 1

회원가입 시 이메일 중복 여부를 확인하고, 입력한 이메일로 발송된 메일을 통해 인증 절차를 거친다.



알람 설정 ☒

닉네임

이름을 입력해주세요.

비밀번호

비밀번호를 입력해주세요.

비밀번호 확인

비밀번호를 입력해주세요.

취소 다음

그림 9. 회원가입 화면 2 - 알람, 닉네임, 비밀번호 입력

사용할 닉네임, 비밀번호를 입력하고, 알람 수신 여부를 선택할 수 있다.



원격 서버 사용자

원격 서버의 사용자명을 입력해주세요.

호스트

원격 서버의 IP 주소를 입력해주세요.

프라이빗 키

SSH pem키를 업로드해주세요.

↶ ↷

취소 다음

그림 10. 회원 가입 화면 3 – 클라우드의 SSH 정보 설정

다음으로, 모니터링할 클라우드 인스턴스에 SSH 접속을 할 수 있도록 사용자명, 호스트, PEM 키 정보를 입력한다.



원격 서버 사용자

ubuntu

호스트

54.180.202.202

프라이빗 키

file:///Users/LLMN/ssh/LLmn.pem

↶ ↷

취소 다음

54.180.244.202
서버는 유효합니다.

완료

그림 11. 회원 가입 화면 4 – SSH 접속 유효성 검사

입력한 SSH 정보를 바탕으로 **실제 접속 테스트**를 진행해, SSH 접속이 정상적으로 작동하는지 확인한다.

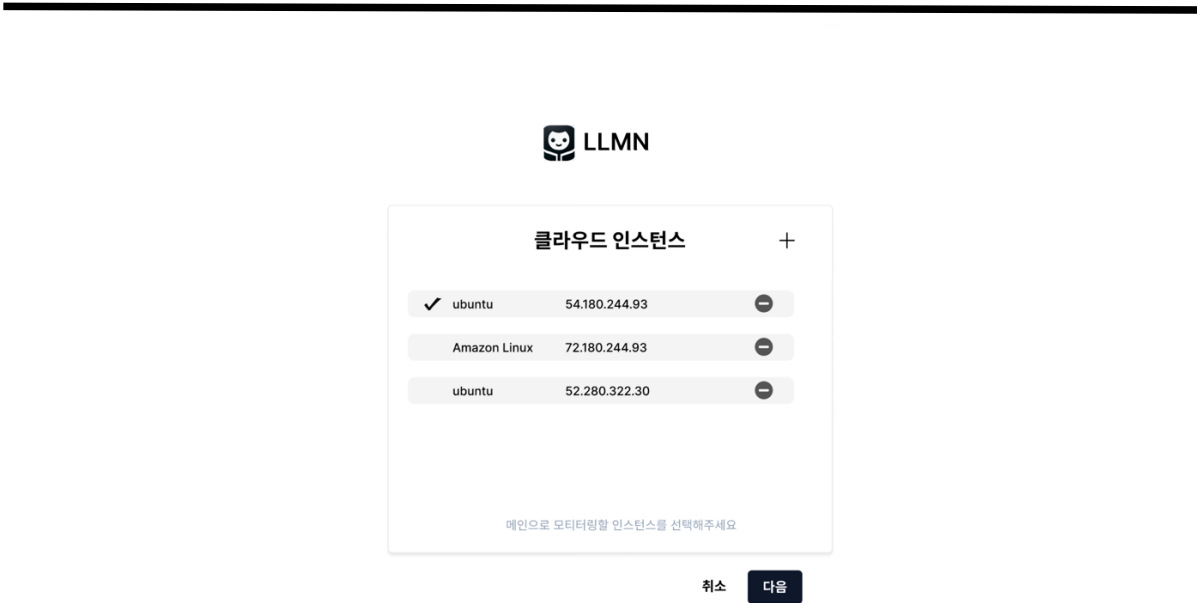


그림 12. 회원 가입 화면 5 – 클라우드 인스턴스 추가

필요에 따라 클라우드 인스턴스를 **추가**할 수 있고, 메인으로 모니터링할 인스턴스를 선택한다. (체크 표시 된 것이 메인으로 모니터링할 인스턴스)

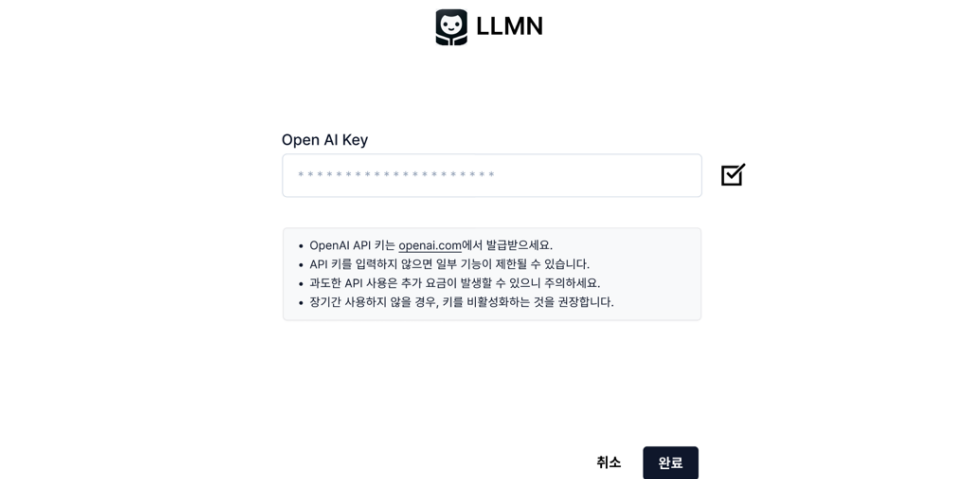


그림 13. 회원 가입 화면 6 – OpenAI API Key 입력

마지막으로, OpenAI API 키를 입력해야 한다. 키는 OpenSSL로 암호화되어 저장되고 입력 후에는 다시 조회할 수 없다.

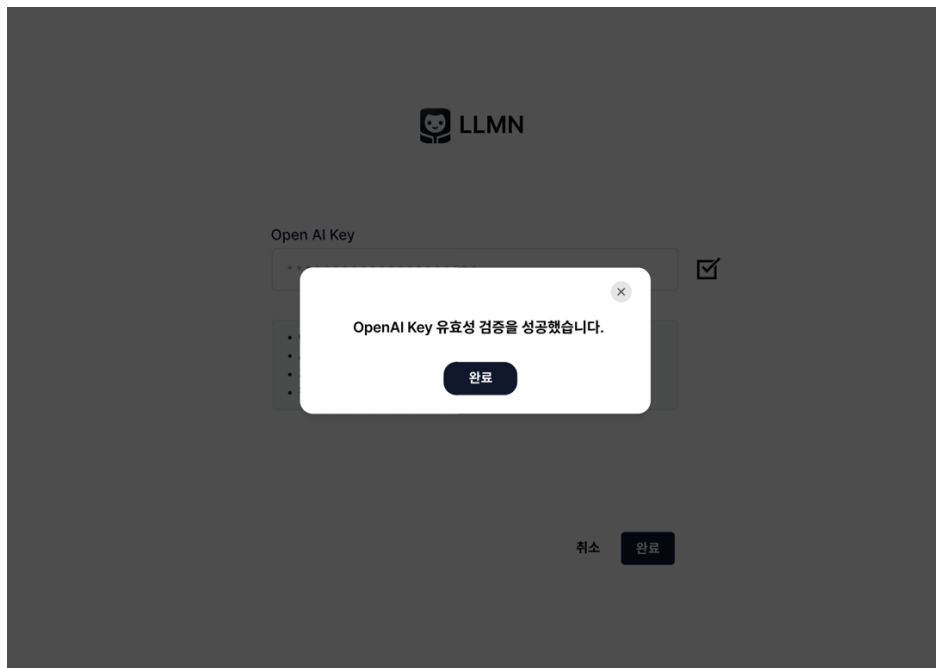


그림 14. 회원 가입 페이지 7 – OpenAI API 작동 유효성 검사

OpenAI API 요청이 정상 작동하는지 확인하고자, 입력된 키를 사용해 **API 요청 검사**를 진행한다.

3.4.4. 로그인



그림 15. 로그인 페이지

회원가입 절차를 완료한 후, 등록된 이메일과 비밀번호를 입력하여 로그인할 수 있다.

3.4.5. 대시보드

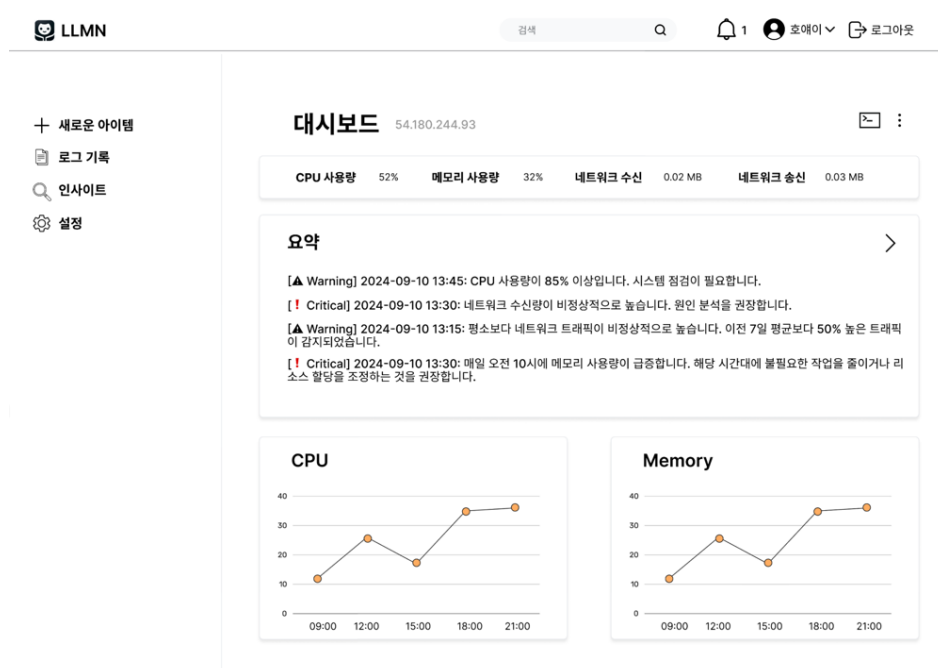


그림 16. 대시보드 화면 1 – 리소스 사용 지표와 핵심 조회

대시보드에서는 모니터링 중인 클라우드의 CPU, 메모리, 네트워크 등의 리소스 사용 지표를 확인할 수 있고, 최근 발생한 이벤트를 요약한 리포트도 확인할 수 있다.

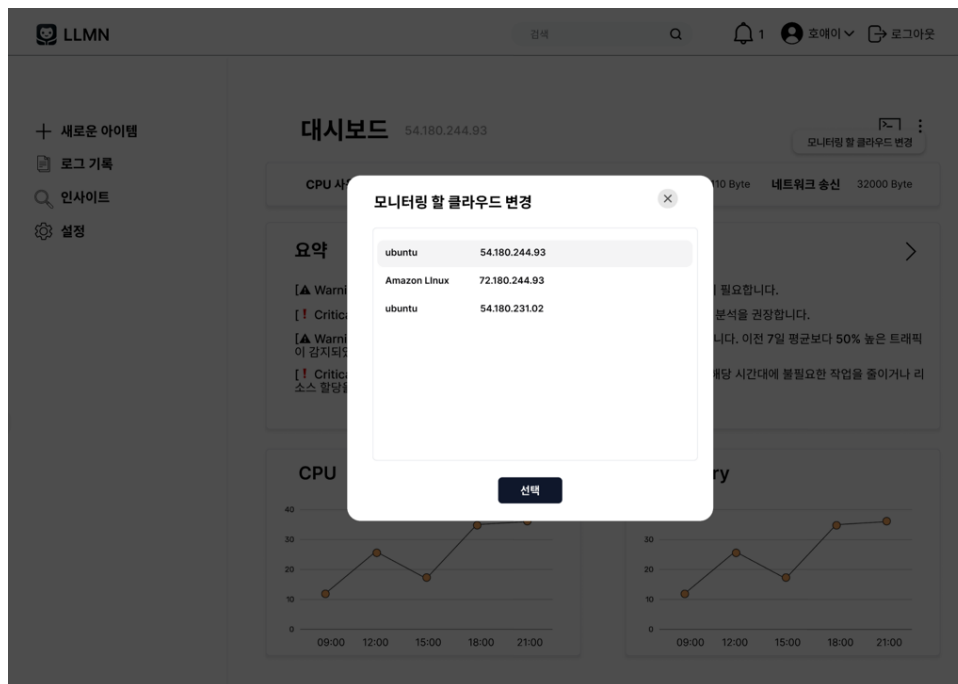


그림 17. 대시보드 화면 2 – 클라우드 변경

필요에 따라 모니터링할 클라우드 인스턴스를 변경할 수도 있다.

3.4.6. 로깅/모니터링

LLMN 검색 1 호메이 로그아웃

+ 새로운 아이템
로그 기록
인사이트
설정

새로운 아이템

<주의> 컨테이너의 이름을 추후에 변경하면 기능이 정상적으로 작동하지 않을 수 있습니다.

프로젝트 이름
이름을 입력해주세요.

설명
설명을 입력해주세요.

클라우드 인스턴스
연결할 인스턴스를 선택해주세요.

컨테이너
연결할 컨테이너를 선택해주세요.

생성

그림 18. 새로운 아이템 화면 – 로깅할 어플리케이션 등록

클라우드 인스턴스 내에 있는 어플리케이션을 아이템으로 등록하면, 해당 어플리케이션의 로그 데이터를 조회하고 분석 리포트를 받을 수 있다.

LLMN 검색 1 호메이 로그아웃

+ 새로운 아이템
로그 기록
인사이트
설정

새로운 아이템

<주의> 컨테이너의 이름을 추후에 변경하면 기능이 정상적으로 작동하지 않을 수 있습니다.

프로젝트 이름
ForPaw - Spring Boot

설명
ForPaw 프로젝트의 스프링 어플리케이션

클라우드 인스턴스
Ubuntu (54.180.244.93)

컨테이너
연결할 컨테이너를 선택해주세요.

Spring
FastAPI
React
연결하지 않음

생성

그림 19. 새로운 아이템 화면 2 – 도커 컨테이너 선택

어플리케이션이 위치한 클라우드 인스턴스를 선택하고, 해당 어플리케이션이 구동되는 컨테이너를 선택하여 등록을 완료할 수 있다.

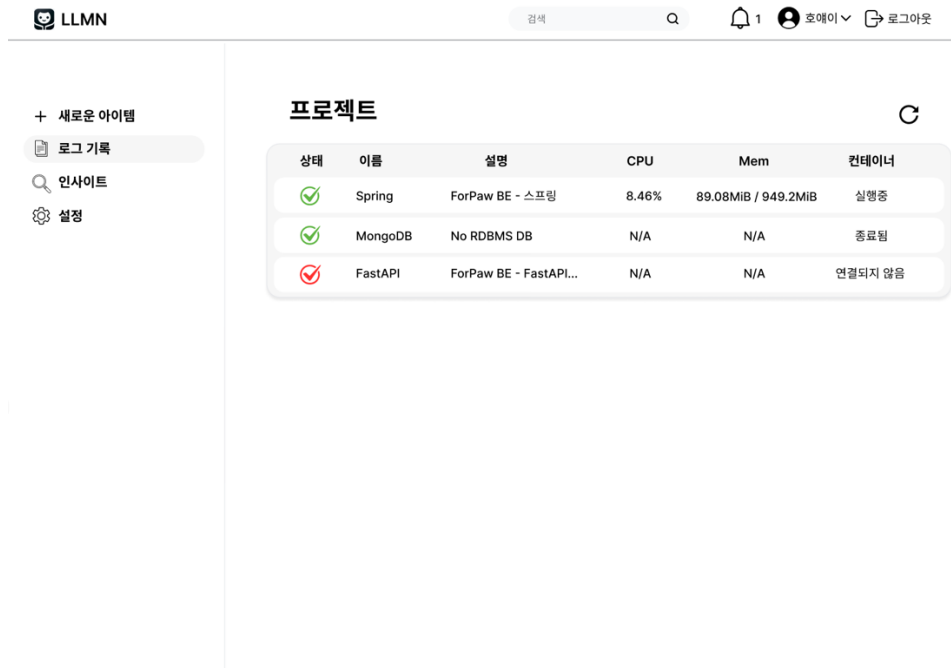


그림 20. 어플리케이션 목록 화면

어플리케이션이 아이템으로 등록되면, 컨테이너의 **자원 사용량**과 **상태**를 실시간으로 확인할 수 있다.

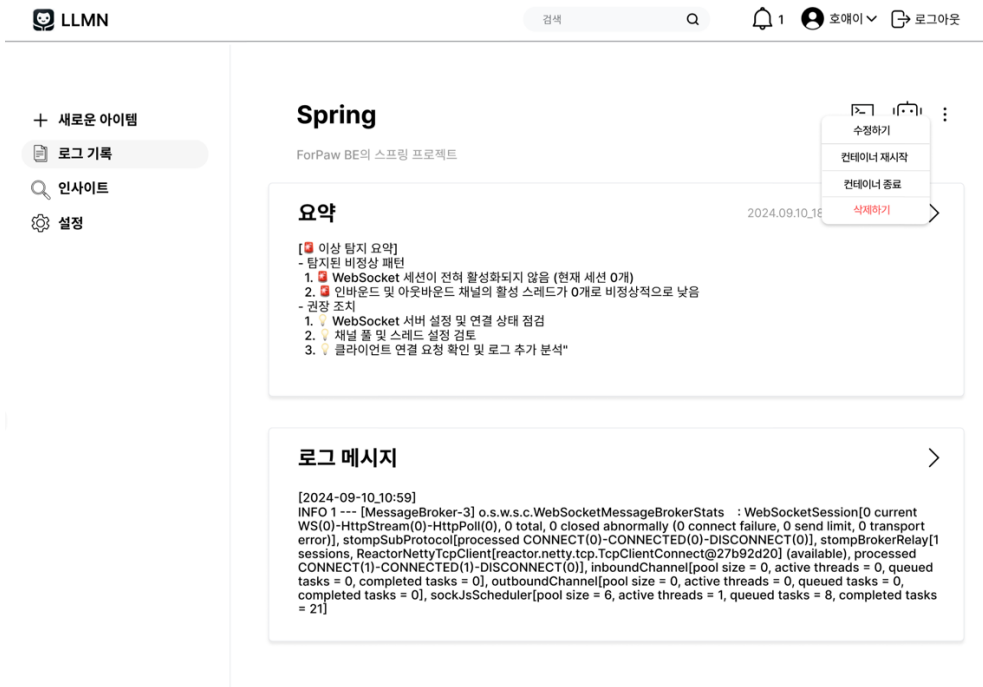


그림 21. 어플리케이션 세부 내용 화면

아이템 세부 페이지에서는 가장 최근의 로그 메시지와 요약 리포트를 조회할 수 있어서, 핵심 이벤트를 빠르게 파악할 수 있다.

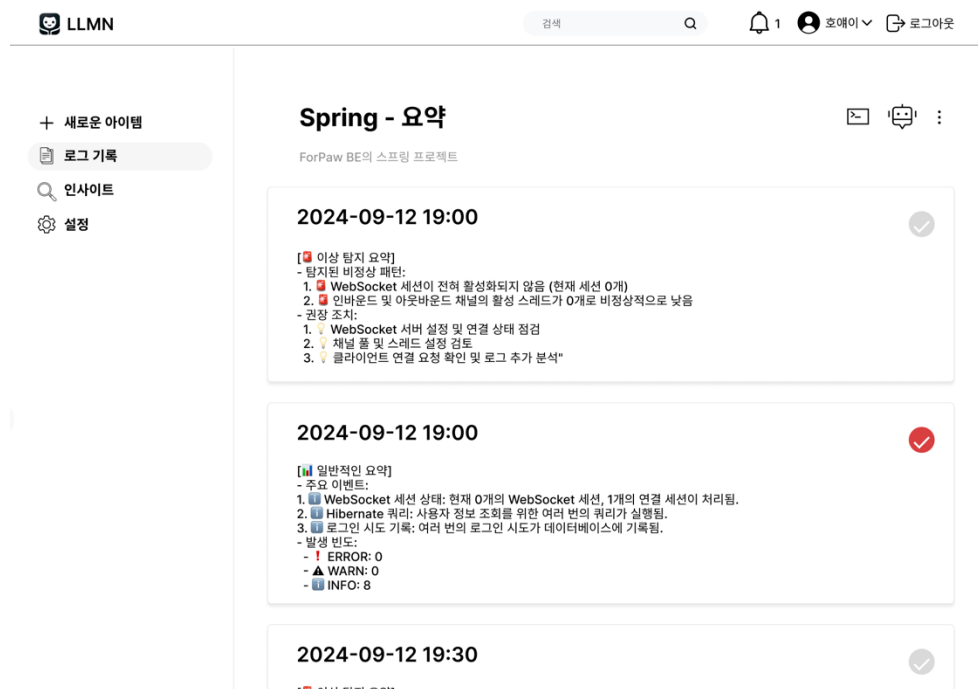


그림 23. 어플리케이션의 요약 목록 화면

또, 시간별 분석 리포트를 조회할 수도 있다.

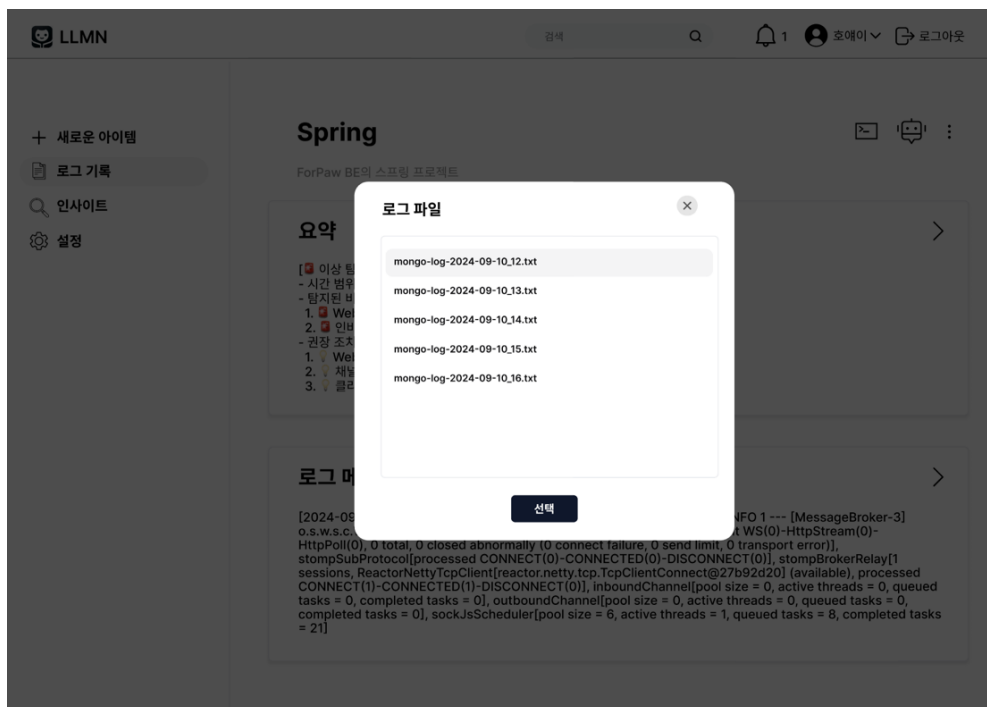


그림 24. 어플리케이션 로그 파일 선택 화면

전체 로그 메시지를 확인하려면, 원하는 로그 파일을 선택해 조회할 수 있다. 이 파일에는 ELK 파이프라인을 통해 주기적으로 수집된 로그 데이터가 저장되어 있다.

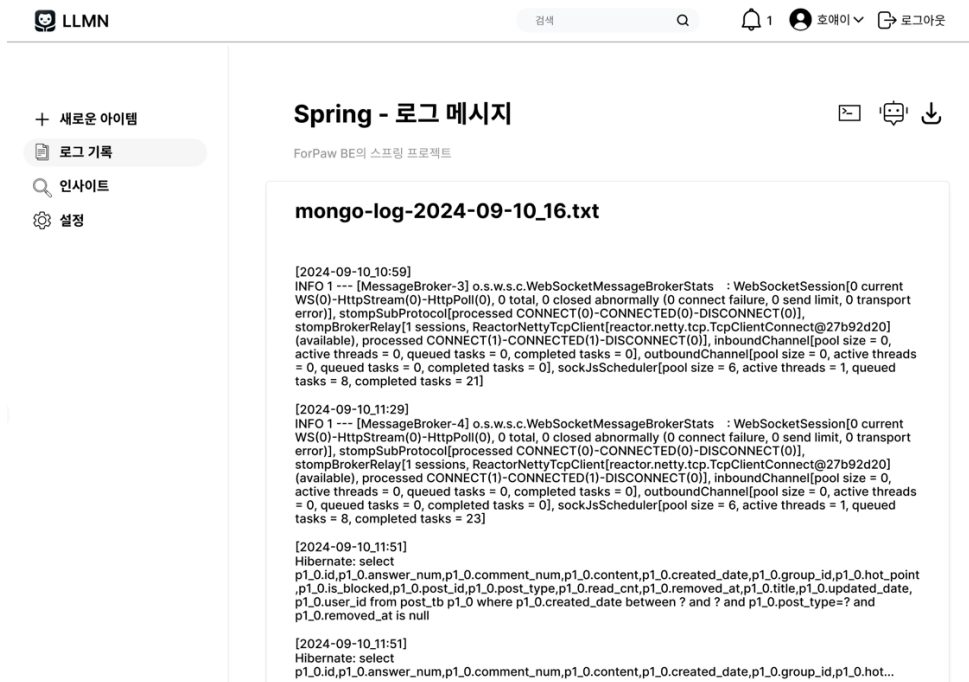


그림 25. 로그 파일 상세 조회 화면

전체 로그 메시지를 조회하는 화면으로, 필요시 해당 로그 파일을 .txt 형식으로 다운로드할 수 있다.

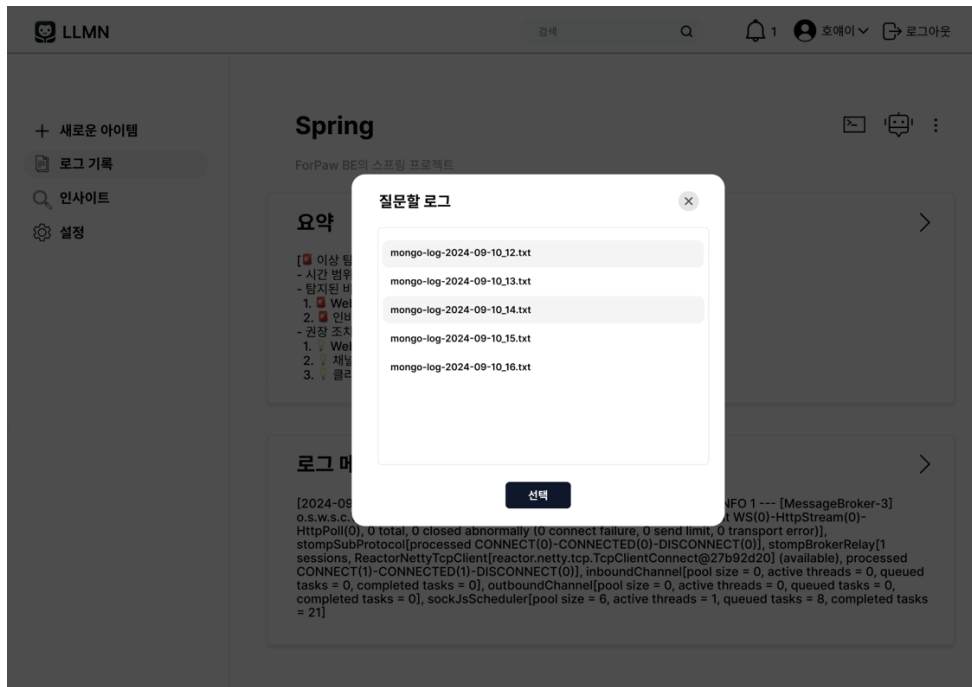


그림 26. LLM에 질문할 로그 파일 선택 화면 (다중 선택 가능)

또한, 질문할 로그를 선택한 후 챗봇을 통해 궁금한 내용을 질문하고, 답변을 받을 수 있다.

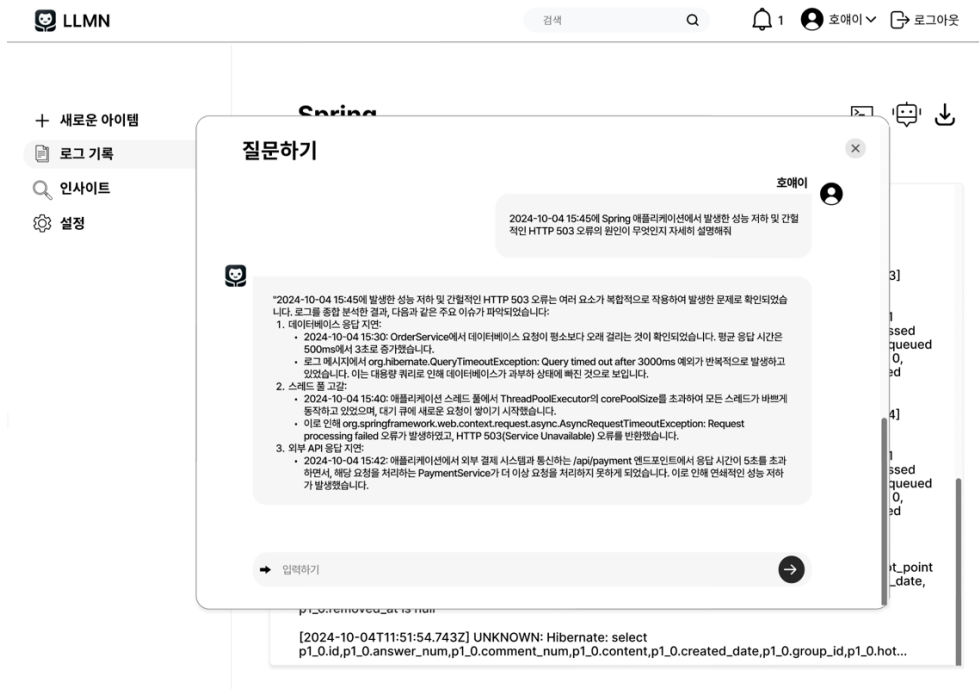


그림 27. LLM과 대화 화면

대화는 실시간 스트리밍 방식으로 진행되고, 문맥 저장 기능 덕분에 지속적인 대화가 가능하다.

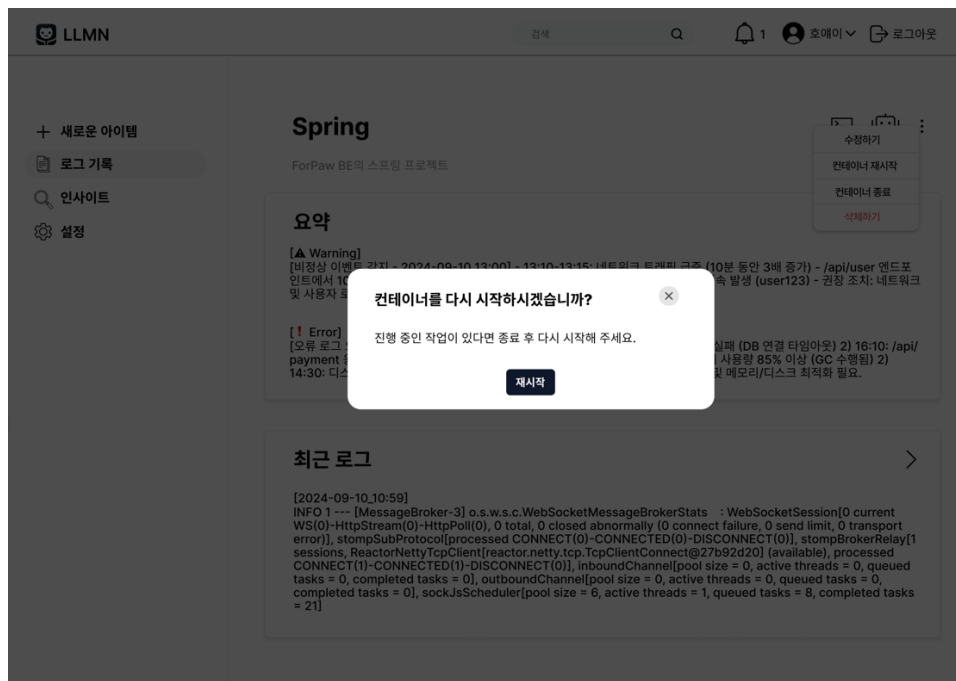


그림 28. 컨테이너 재시작 요청 화면

필요한 경우 도커 컨테이너를 재시작하거나 종료할 수 있어서, 관리자가 즉각적으로 대응할 수 있다.

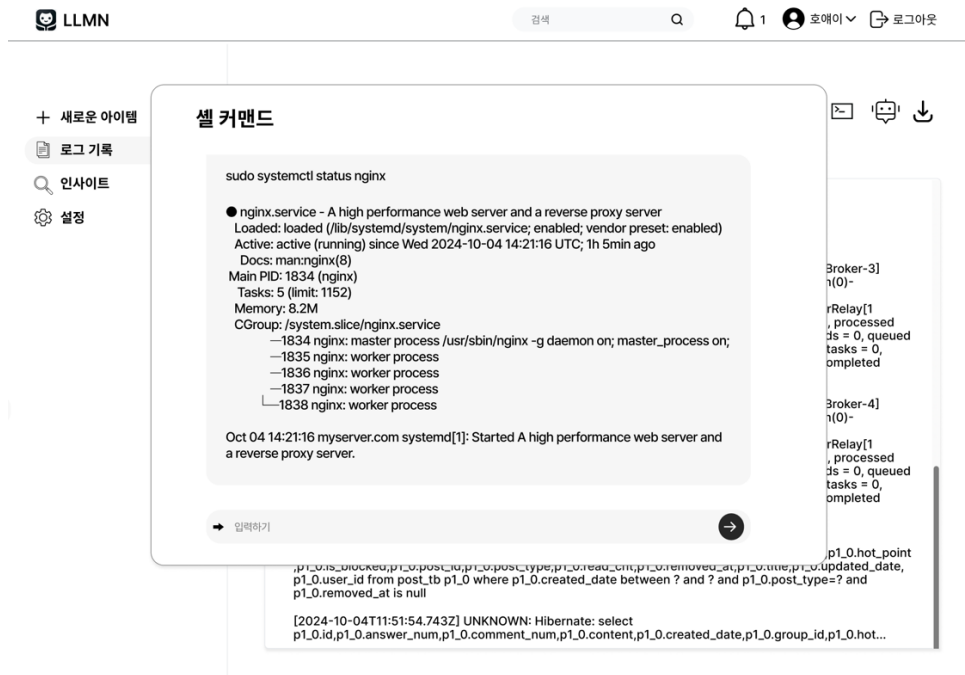


그림 29. 셸 커맨드 화면

구체적인 명령어 입력이 필요할 때는, 프롬프트 아이콘을 클릭하여 명령어를 직접 입력할 수 있다. 내부적으로 Apache SSHD를 이용한 SSH 연결을 통해, 클라우드 인스턴스에 셸 명령어를 전송하고 응답을 받을 수 있도록 구현되어 있다.

3.4.7. 인사이트

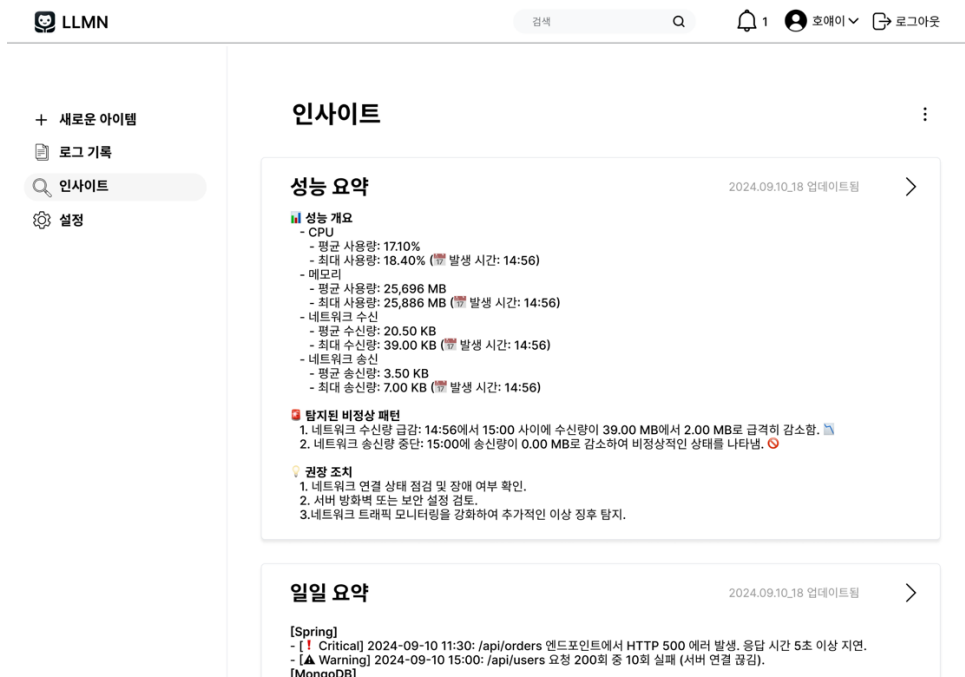


그림 30. 인사이트 목록 화면 1

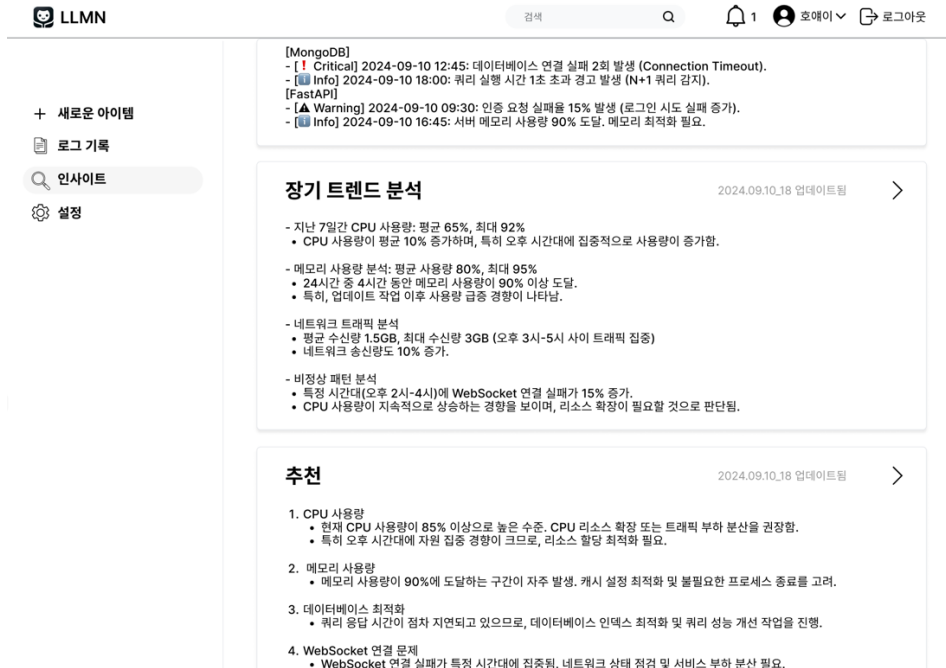


그림 31. 인사이트 목록 화면 2

인사이트 페이지에서는 LLM을 통해 생성된 리포트를 확인할 수 있으며, '성능 요약', '일일 요약', '장기 트렌드 분석', '추천'의 네 가지 유형의 리포트를 제공한다.

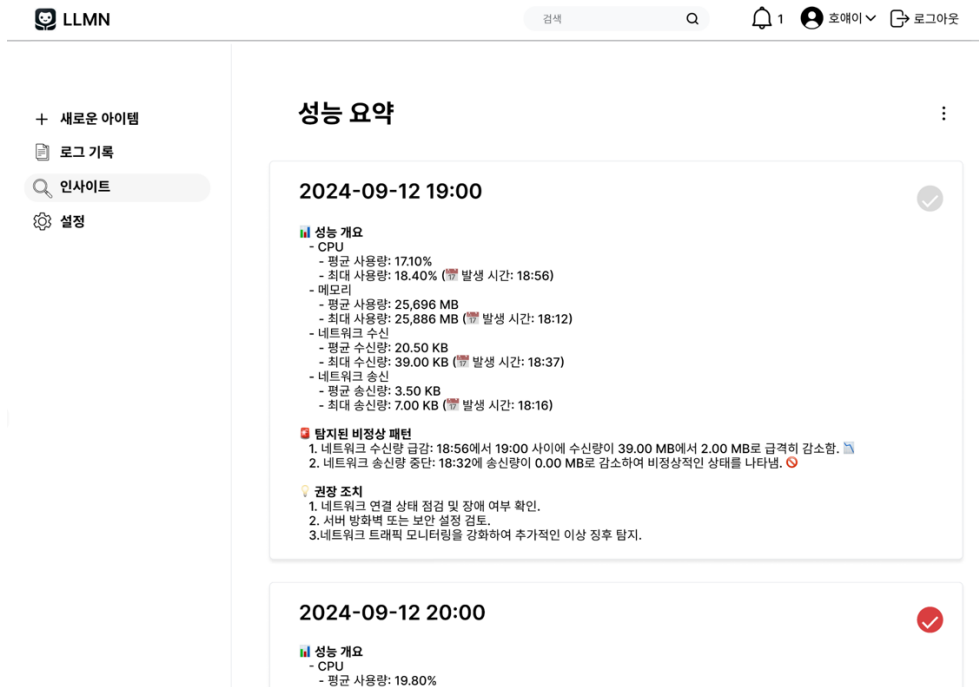


그림 32. 성능 요약 상세 화면

각 리포트의 상세 페이지로 이동하면, 시간별로 분석 리포트를 확인할 수 있다.

3.4.8. 검색

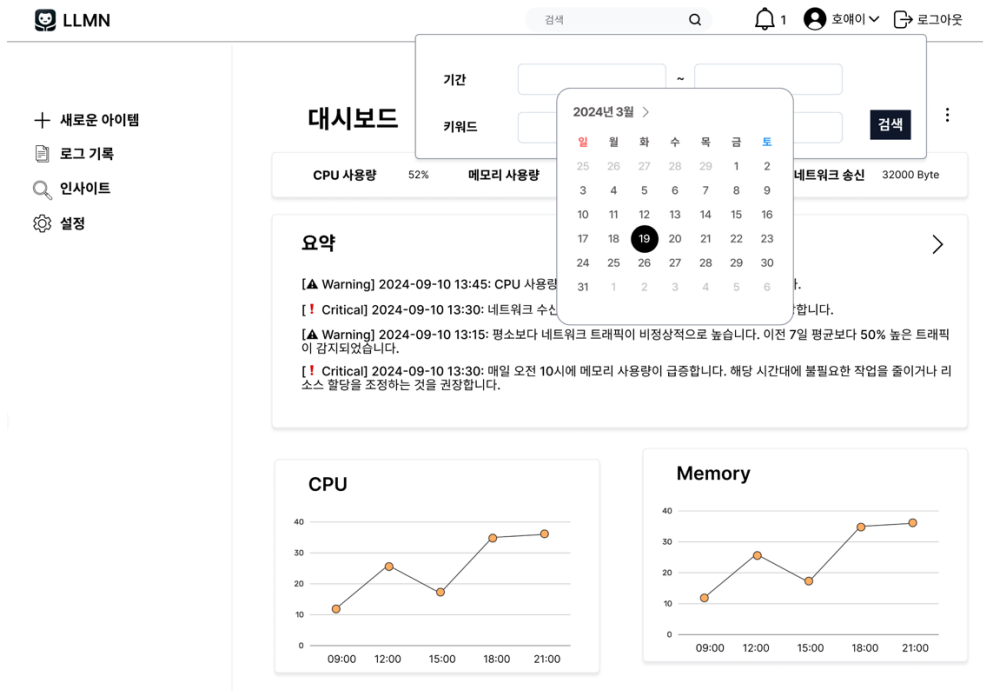


그림 33. 검색 화면 – 기간, 키워드 설정

어플리케이션의 로그 파일이나 분석 리포트를 **빠르게** 찾고자 할 때는, 검색 기능을 활용 할 수 있다.

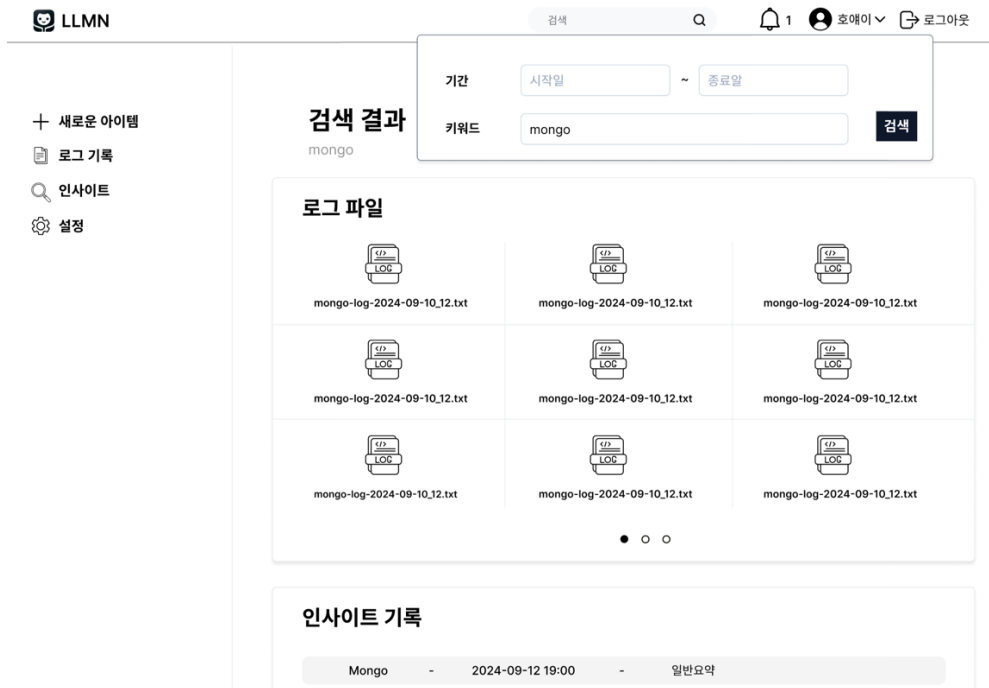


그림 34. 검색 결과 화면 1

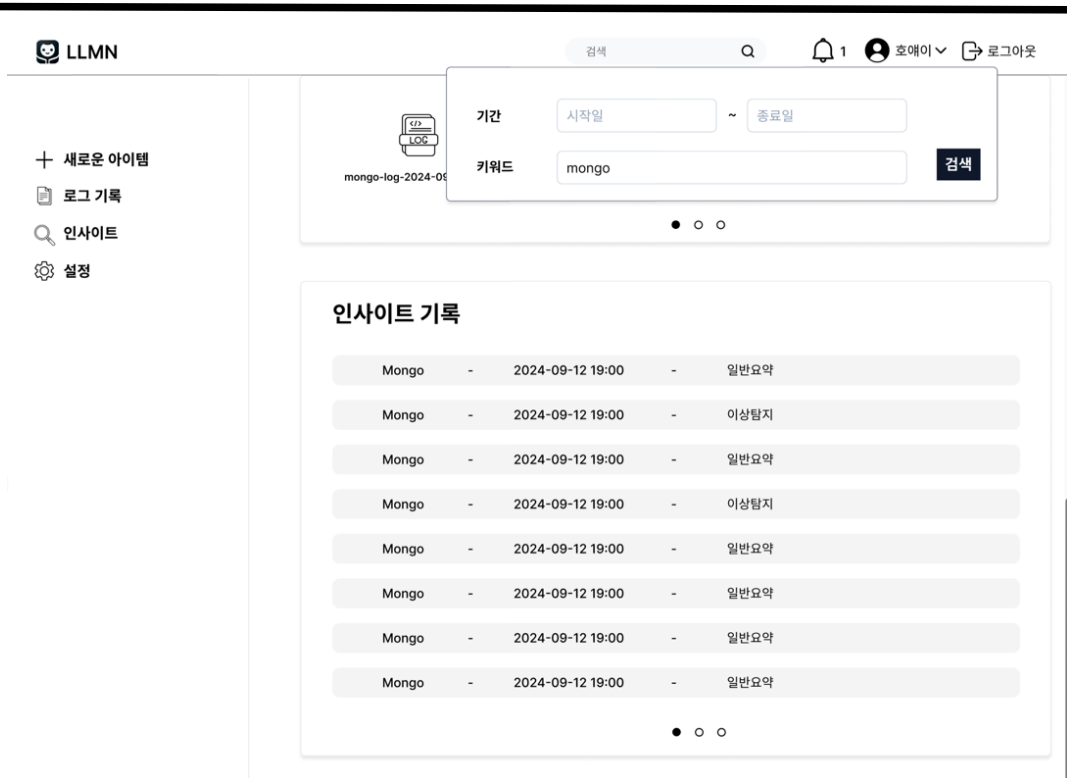


그림 35. 검색 결과 화면 2

기간과 키워드를 입력해 검색하면, 결과가 목록으로 표시되고, 원하는 로그 파일이나 리포트를 클릭해 내용을 확인할 수 있다.

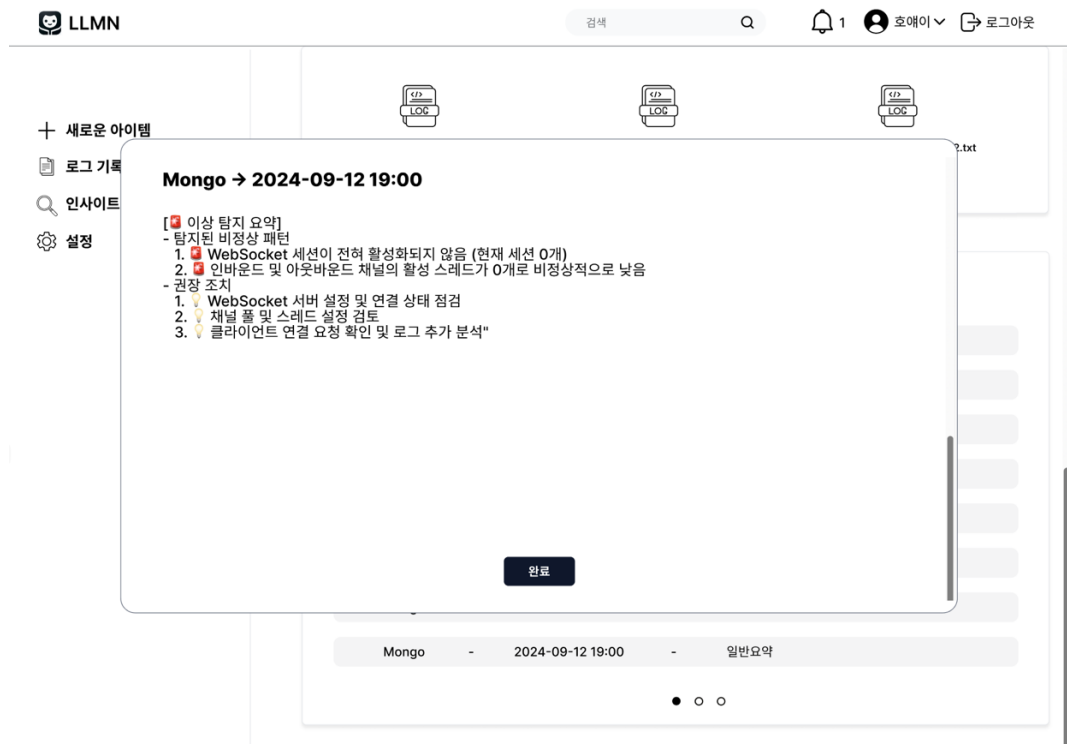


그림 36. 검색 결과 화면 3

3.4.9. 알람

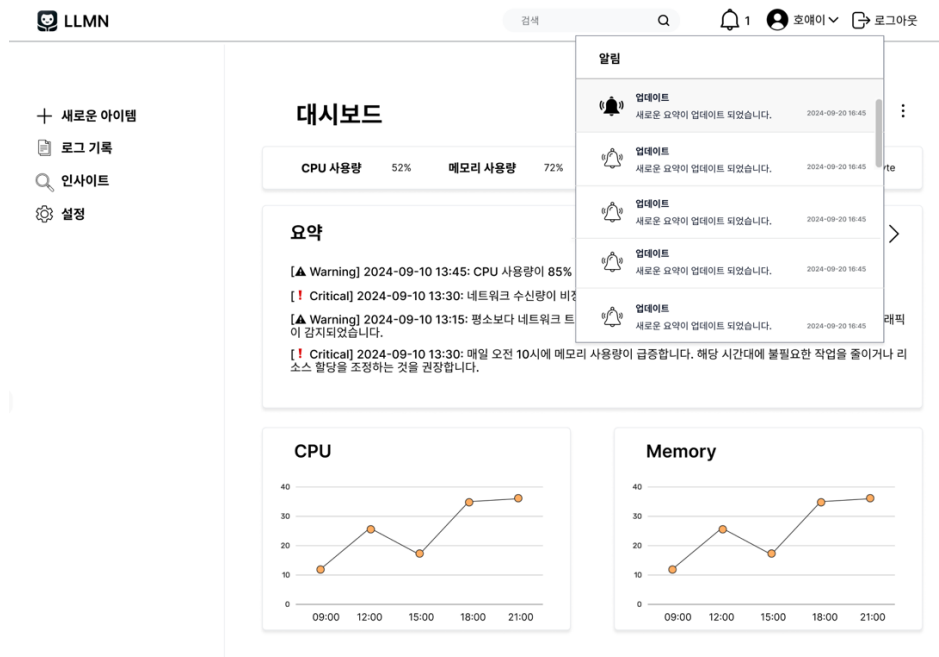


그림 37. 알람 조회 화면

새로운 분석 리포트가 생성되거나 **위협 요소**가 감지되면, 상단의 알람 아이콘을 통해 알람을 확인할 수 있다.

3.4.10. 설정

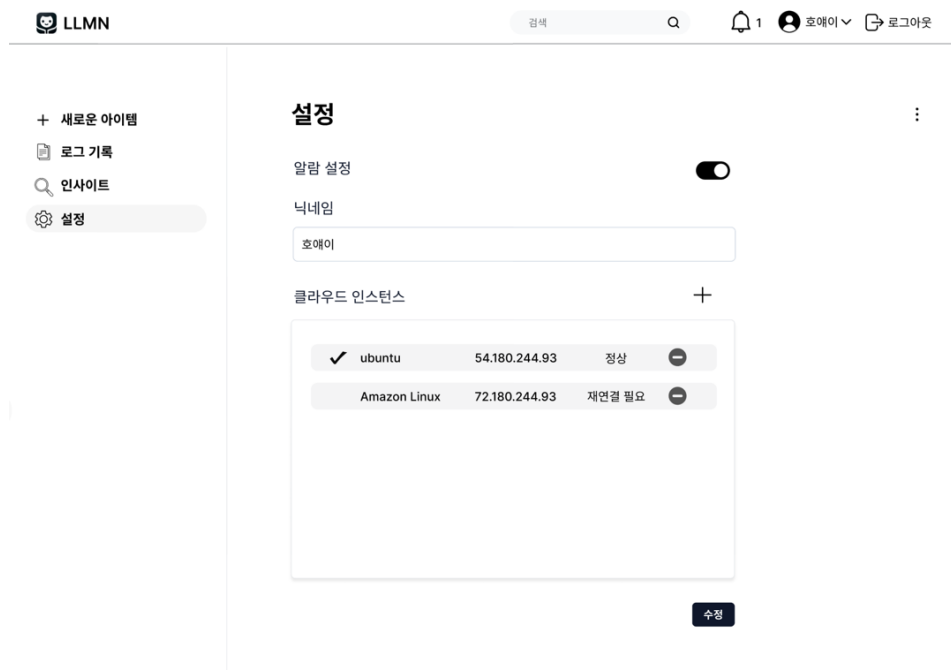


그림 38. 설정 화면 1 – 알람, 닉네임, 클라우드 설정

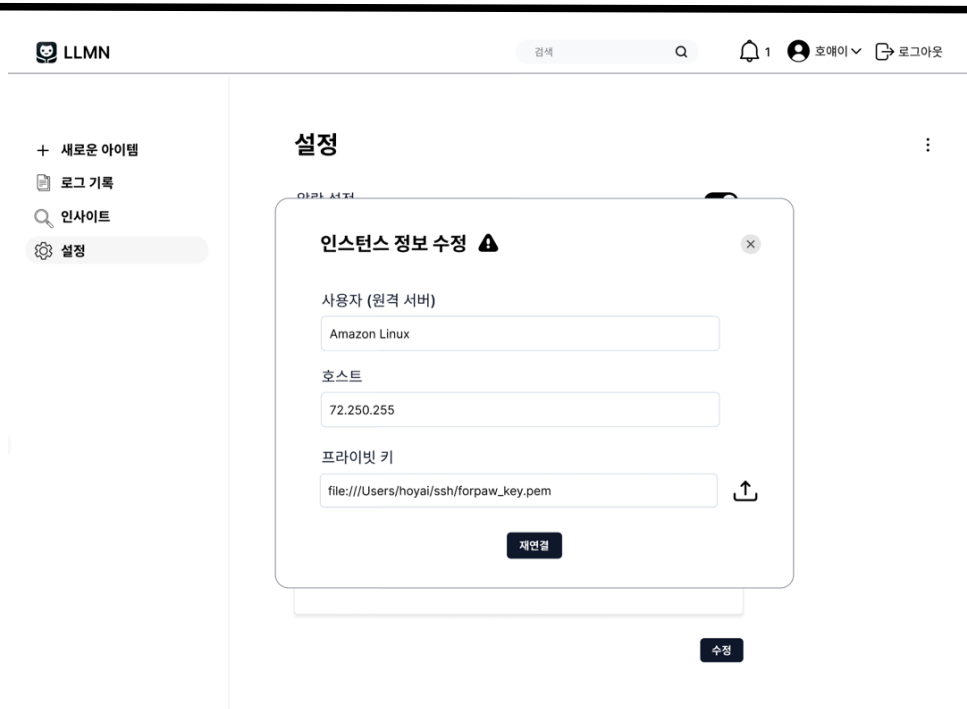


그림 39. 설정 화면 2 – 인스턴스 정보 수정

설정 페이지에서는 알람 수신 여부, 닉네임, 클라우드 SSH 정보를 수정할 수 있다. 또 연결 실패한 클라우드 인스턴스에 대해 재연결을 시도할 수도 있다

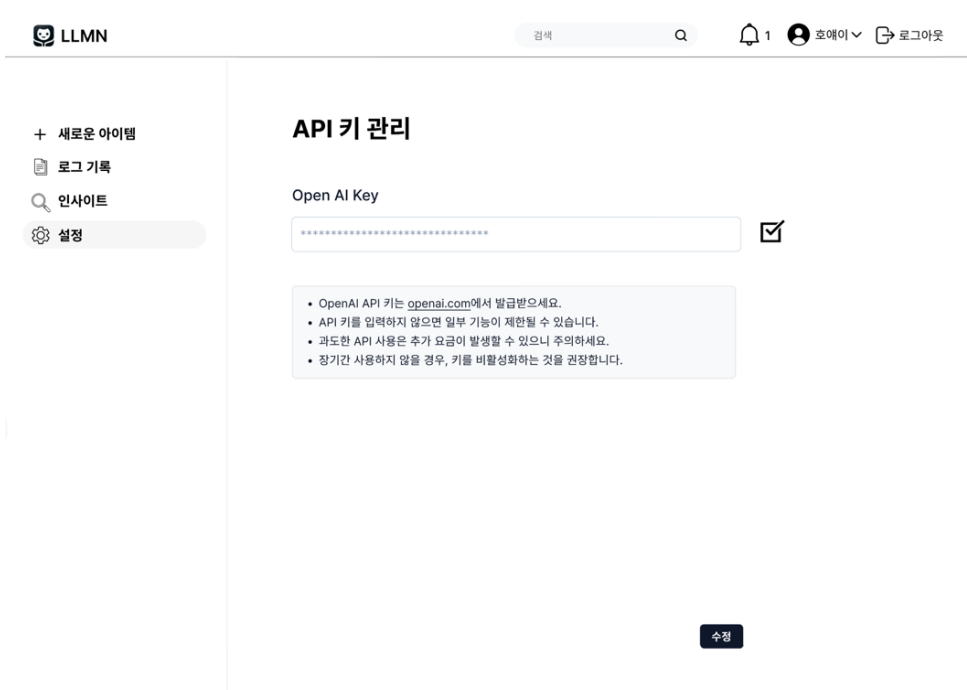


그림 40. 설정 화면 3 – API 키 재등록

그리고, 필요에 따라 OpenAI API 키를 새로 등록 하여 보다 안전하게 관리할 수 있다.

4. 연구 결과 분석 및 평가

4.1. 실시간 로그 수집

본 시스템은 Filebeat와 Logstash를 사용하여 여러 클라우드 인스턴스에서 실시간으로 로그 데이터를 수집하고, 이를 Elasticsearch에 저장하여 분석하는 기능을 구현하였다. 이를 통해 다양한 로그를 **일관성** 있게 처리할 수 있었고, 관리자는 시스템 상태를 **신속**하게 모니터링할 수 있었다. 그 결과, 운영 중 발생하는 이벤트에 즉각적으로 대응하는 것이 가능해졌다.

그러나, 다수의 인스턴스에서 실시간으로 로그를 수집하면서 시스템 부하가 증가하여, 다른 서비스에 영향을 미치는 경우가 있었다. 이를 해결하기 위해, 로깅 인프라를 분산 처리 방식으로 전환하거나, 수집 프로세스에 큐를 도입해 시스템 부하를 효과적으로 관리하는 것이 필요하다.

4.2. LLM을 활용한 로그 요약 및 이상 패턴 탐지

LLM을 활용하여 로그 요약 및 이상 패턴 탐지 기능을 구현한 결과, 시스템 운영의 효율성이 크게 향상되었다. **긴 로그** 파일에서도 **핵심 정보**를 빠르게 추출하고, 비정상 패턴을 **효율적**으로 탐지할 수 있어서 관리자가 시스템 문제를 보다 신속하게 인식하고 대응할 수 있었다.

다만, 매우 복잡하거나 방대한 로그 데이터를 처리할 때, 일부 중요한 정보가 누락되거나, 요약된 내용이 다소 모호하게 표현되는 문제가 있었다. 이는 프롬프트 설계가 충분히 정교하지 않거나, 파인 튜닝이 부족했기 때문으로, 이 부분을 개선하기 위한 추가적인 연구가 필요하다.

4.3. 원격 클라우드 SSH 접속

Apache MINA SSHD를 활용한 원격 SSH 접속 기능을 통해 관리자는 서버에 **직접 접속하지 않고도** 시스템 상태를 실시간으로 점검하고, 원격으로 명령을 실행할 수 있었다. 특히, 시스템이나 애플리케이션에 문제가 발생할 경우, **즉각적**으로 필요한 조치를 취해 시스템의 가용성과 안정성을 효과적으로 유지할 수 있었다.

4.4. 웹 기반 클라이언트 환경

React, Next.js, TailwindCSS로 구현된 웹 인터페이스는 관리자가 시스템을 직관적으로 모니터링하고 빠르게 대응할 수 있는 환경을 제공하였다. 이를 통해 사용자는 로그 데이터를 간편하게 조회하고, 분석 결과를 시각적으로 확인하여, 시스템 관리를 보다 직관으로 수행할 수 있었다. 추가로, 반응형 디자인을 적용해 다양한 기기에서 일관된 사용자 경험을 제공하고 접근성을 높였다.

4.5. LLM 기반 로그 분석 비용 평가

OpenAI의 GPT-4o-mini API를 사용하여 로그 데이터를 요약하고 이상 패턴을 탐지한 결과, **비용 효율성** 측면에서도 긍정적임을 확인하였다.

웹 서비스를 운영하는 동안 하루 평균 약 1달러 미만의 비용이 발생하였으며, 이는 저비용으로도 로그 분석 및 요약 기능을 사용할 수 있음을 보여준다. 이러한 결과는 향후 더 큰 규모의 운영 환경에서도 충분히 확장 가능한 가능성을 보여준다.

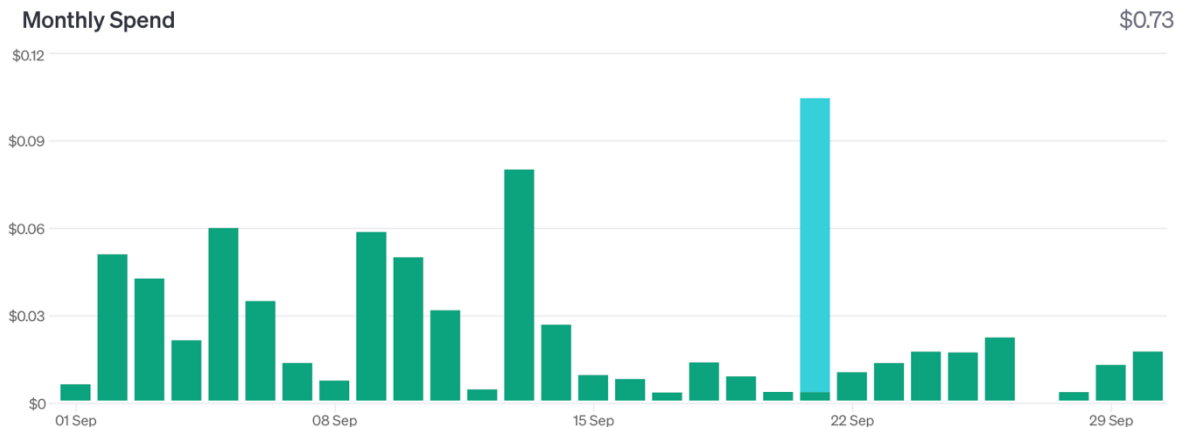


그림 41. GPT-4o-mini 비용

5. 결론 및 향후 연구 방향

본 졸업 과제에서는 다양한 클라우드 인스턴스에서 수집한 로그를 **실시간**으로 분석하고, 이를 활용하여 시스템 상태와 로그 요약 결과를 관리자에게 제공하는 시스템을 개발하였다.

LLM을 적용한 로그 분석 시스템은 로그를 요약하고 이상 패턴을 탐지하는 기능을 제공하여, 로그 분석의 효율성을 크게 개선하였다. 뿐만 아니라, 관리자가 실시간 성능 지표를 모니터링하고, SSH를 통해 **원격 서버**를 제어할 수 있게 하여, 문제 발생 시 신속하게 대응할 수 있는 환경을 마련하였다

이 시스템은 단순한 로그 모니터링을 넘어, **중요한** 정보를 **직관적**이고 **신속**하게 제공하여 관리자의 실시간 대응 능력을 향상시킨다. 특히, 대규모 인프라를 운영할 때, 로그 데이터에서 핵심 정보를 빠르게 추출하고 파악하는 것은 시스템 중단을 방지하고 운영 안정성을 극대화하는 데 필수적이기에 매우 중요한 역할을 한다.

향후 연구 방향은 현재의 로그 분석 시스템을 더 고도화하는 데 중점을 둘 계획이다. 이미 시간별 로그 요약을 기반으로 일일 요약물을 생성하고, 이를 종합하여 장기적인 트렌드 분석을 도출하는 기능이 구현되어 있으나, 이를 더욱 세밀하게 다듬어 기능의 완성도를 높일 것이다.

또한, 요약 과정에서 **필터링**을 더욱 정밀하게 적용하여, 핵심 정보와 반복적인 패턴을 더욱 정확하게 구분할 수 있도록 개선할 것이다. 이를 통해 불필요한 정보를 배제하고, 관리자에게 보다 유의미한 데이터를 제공함으로써 로그 관리의 효율성을 극대화 시킬 것이다.

마지막으로, 관리자에게 **권장 조치**를 **제안**하는 기능도 한층 강화할 것이다. 현재 비정상적인 동작을 감지하고 권장 조치를 제시하는 기능을 갖추고 있으나, 추후에는 상황에 맞춰 더욱 구체적인 대응 방안을 제시할 수 있도록 기능을 개선할 것이다. 예를 들어, 성능 저하나 오류가 발생한 경우, 시스템이 관리자에게 상황에 맞는 **단계별 대응 방안**을 제시하여, 보다 신속하고 효율적인 문제 해결이 가능하도록 할 예정이다.

6. 구성원별 역할

팀원	역할
이한홍	- 착수, 중간, 최종 보고서, 포스터 작성 설계 - REST API 설계 및 데이터 베이스 설계 - Figma를 활용한 UI/UX 설계 BackEnd (Spring Boot) - Filebeat, Logstash, ElasticSearch을 연동한 로그 수집 파이프라인을 구축 및 Spring Boot와 통합 - Apache SSHD로 클라이언트와 클라우드 서버 간 SSH 연결 기능 구현 - Redis Pub/Sub을 활용한 실시간 명령 전송 및 응답 처리 구현 - 로그인, 회원관리, 프로젝트 관리, 성능 모니터링, 로그 분석 및 알림 API 개발 BackEnd (FastAPI) - LangChain과 OpenAI API 연동, 프롬프트 작성 및 로그 분석 API 개발 - SSE(Server Sent Events) 기술로 LLM의 응답을 실시간 스트리밍 형식으로 제공하는 기능 구현
이종일	FrontEnd - React, NextJS, TS, TailWind CSS를 이용한 웹 페이지 구현 - UI/UX 개선을 통한 반응형 웹 페이지 구현

	- SSR(서버 사이드 렌더링) 도입으로 실시간 데이터 처리 구현
박재홍	- 로그 분석을 위한 LLM 프롬프팅 작성에 기여 - 시연회 준비 및 발표 자료 제작

7. 일정

6월					7월				8월					9월				10월		
1	2	3	4	5	1	2	3	4	1	2	3	4	5	1	2	3	4	1	2	3
요구 사항 분석																				
		아키텍처 설계																		
				API 및 DB 설계																
						UI/UX 설계														
								중간 보고서												
									ELK 파이프라인 구축											
										LangChain 구축										
														Spring Boot API 개발						
														Fast API API 개발						
															FronD-End 개발					
																	배포			
																		최종 보고서		

8. 멘토 의견 반영

8.1. 컨셉과 시나리오가 여전히 구체적이지 않음

- 멀티 클라우드 환경에서 각 인스턴스의 로그 데이터를 실시간으로 수집하고 분석하는 LLM 기반 시스템으로 컨셉을 명확히 설정하였다.

- LLM 을 활용해 로그 데이터를 시간대별로 **요약**하고 실시간으로 비정상 패턴을 **감지**하여 주요 이벤트를 빠르게 **파악**할 수 있도록 함으로써, 관리자가 신속히 **대응**하고 보안과 운영에 필수적인 의사결정을 지원하는 구체적인 시나리오를 구성하였다.

8.2. 범용성 있는 로그 분석 시스템 개발에 한계가 있을 것임

- 다양한 로그 형식을 일관되게 처리하기 위해 Filebeat 와 Logstash 를 사용해 로그 데이터를 통합한 뒤, 공통된 구조로 변환하여 Elasticsearch 에 저장하도록 설계하였다. 이를 통해 **개별** 서비스의 형식에 구애받지 않고, **통일된 구조**로 로그 데이터를 **일관되게** 분석할 수 있다.

- 광범위한 데이터 학습으로 **범용성**을 갖춘 LLM 을 활용하여 로그 요약과 이상 패턴 탐지 기능을 구현함으로써, 특정 서비스에 **종속**되지 않고도 다양한 애플리케이션의 로그 데이터를 일관되게 분석할 수 있다. 또한, 범용성을 **유지**하면서도 로그 데이터를 **세밀**하게 분석할 수 있도록 정교한 프롬프팅을 적용하여, 고도화된 분석이 가능하도록 하였다.

8.3. 타겟을 명확히 설정한 후 개발 진행 필요

- 주요 타겟 사용자를 대규모 **클라우드 환경**에서 **인프라 운영**을 담당하는 시스템 관리자로 설정하였다. 이들은 다양한 클라우드 환경에서 발생하는 로그 데이터를 실시간으로 모니터링하고, 시스템 운영 상태를 점검하며, 보안 위협을 탐지하는 기능이 필요로 한다.

9. 참고 문헌

- [1] Josh Long, Kenny Bastani, "Cloud Native Java: Designing Resilient Systems with Spring Boot, Spring Cloud, and Cloud Foundry", O'Reilly Media, 2017.
- [2] Daniel J. Barrett, Richard E. Silverman, Robert G. Byrnes, "SSH, The Secure Shell: The Definitive Guide", O'Reilly Media, 2005.
- [3] Navin Sabharwal, "Application Observability with Elastic: Real-time Metrics, Logs, Errors, Traces, Root Cause Analysis, and Anomaly Detection", Apress, 2022.